T Tutorial Lab 12.2 (Formatively Assessed):

FET Terminal Characteristics and a Single-stage FET amplifier

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

5. Open a terminal window and ensure that you are in your home directory using the following command:

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

6. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules
module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

a. ***Note: This step only needs to be completed ONCE - If you already ran this script in lab 12.1, you don't need to run it again here.*** The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.

7. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD
virtuoso &
```

Note: The 'virtuoso &' command starts Cadence. The ampersand symbol (&) causes the Cadence process to run in the background freeing up the terminal for other commands the '&' symbol is a feature of the terminal's shell (bash) not Cadence.

1. Learning Objectives of this Lab:

From the previous lab, you should know how to:

- Start a remote Linux Lab session using the X2go remote desktop client.
- Set up the Linux terminal environment for Cadence and launch the Command Interpreter Window (CIW).
- Open the Library Manager. Understand the hierarchy of libraries, cells, and views. Identify the four main libraries used in this course (Designs, analogLib, basic, and gpdk180) and their uses. (cdsDefTechLib and subtrateLib can be ignored.)
- Create, edit, check, and save a new schematic using the Library Manager and the Virtuoso Schematic Editor. Instantiate components from the analogLib library. Edit component properties before and after instantiation. Move, drag, stretch, rotate, and flip components before and after instantiation. Delete components. Zoom and pan the schematic editor canvas. Understand the role of nets and nodes. Add wires and create wire names.
- List the four basic SPICE analysis types (transient, DC, AC, Stability).
- Launch the Analogue Design environment (ADE). Set up a transient analysis. Simulate a schematic using the ADE. Plot the voltage and current waveforms at given nets and nodes.

In this lab, you will:

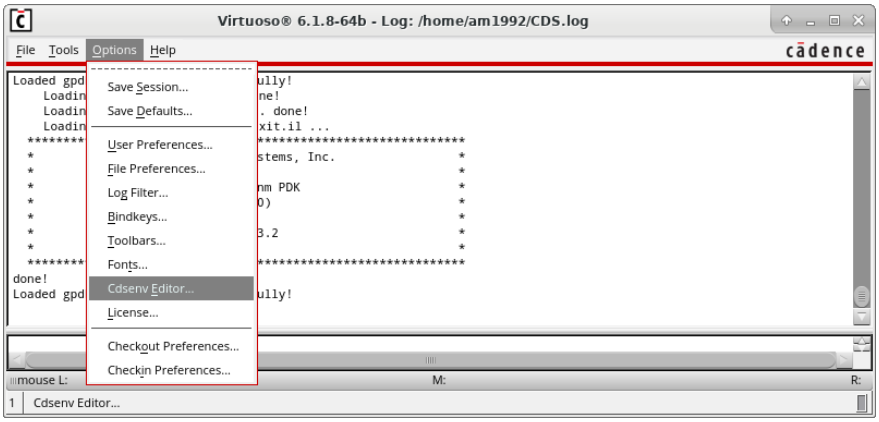
- Instantiate and characterise an NMOS FET from the GPDK180 library.
- Create and simulate a simple resistor-loaded single-stage NMOS FET voltage amplifier.
- Address the inadequacies of using a resistive load, and implement a current-source loaded NMOS FET voltage amplifier.

2. Cadence Documentation, Help, and Licensing

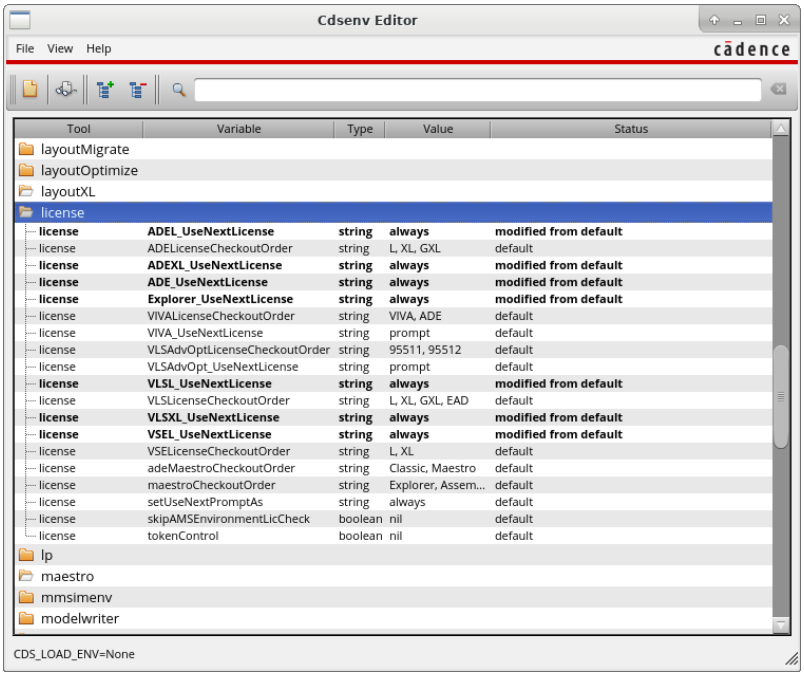
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note all licenses should be set to 'always' >

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(1) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(2) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use
`clsAdminTool -help`

4. Circuit Simulation Exercises: Part 1

With guidance, you will:

- Instantiate and parameterise an NMOS FET from the GPDK180 library
- Create and simulate a test circuit for the evaluation of the NMOS FET V_{GS}-I_{DS} and V_{DS}-I_{DS} terminal characteristics.

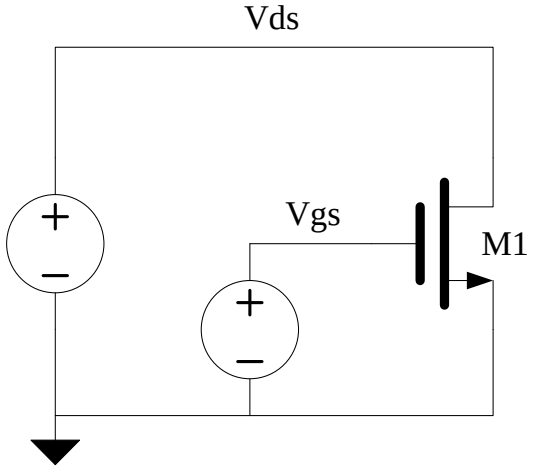
The ideal MOSFET is typically modelled as a Voltage-Controlled Current-Source (VCCS); by controlling the voltage at the input—the gate-to-source voltage V_{GS}—we can control the current that flows at the output—the drain-to-source current I_{DS}.

Here, a distinction must be made between the large-signal nonlinear relationship of V_{GS} and I_{DS}, and the small-signal (idealised, linearised) relationship of the same variables. The small-signal transconductance gm of a MOSFET relates to its large-signal terminal characteristics by $\mathrm{gm}(V_{GS}) = \mathrm{d}I_{DS}/\mathrm{d}V_{GS}$. Further, the non-ideal MOSFET deviates from the VCCS model by a dependency of the output current I_{DS} on the output voltage V_{DS}. This non-ideal behaviour is modelled in the small-signal domain by a parasitic resistance r_o between the output terminals of the VCCS, where $r_o = \mathrm{d}V_{DS}/\mathrm{d}I_{DS}$.

In this lab, we will analyse the MOSFET in the large-signal domain; that is, we will consider the input-output relationship of the MOSFET for a large, full-scale swing of the input signal. We will characterise – through simulation – the V_{GS}-I_{DS} and V_{DS}-I_{DS} large-signal terminal behaviour of an NMOS device from the GPDK180 library. In the following lab (12-3) we will relate these large-signal characteristics to the small-signal model of the NMOS device.

4.1 NMOS Terminal Characteristics

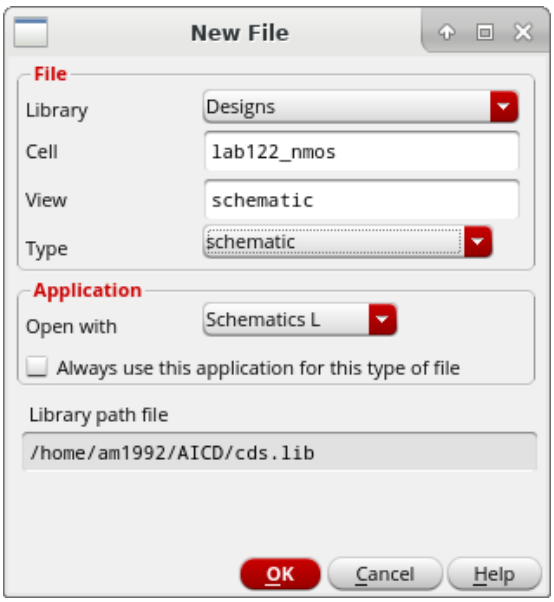
(4.1.1) Create the schematic for the NMOS testbench circuit in Figure 1. A summary of the circuit's operation is given in section 4.1 of the background material.



< Figure 30. Circuit for Large Signal Transconductance Terminal Characteristics Evaluation. Reproduced from Figure 14, section 4.1 of the background material. >

(4.1.2) From the Library Manager window, select the 'Designs' library and choose **Menu: Files à New à Cell View** to invoke the New File form (Figure 2).

In the 'Cell' text field, choose a meaningful cell name (e.g., lab122_nmos). In the 'View' text field, type 'schematic' as the view name and from the 'Type' drop down menu choose 'schematic' as the view type. In the 'Application' section of the dialog box, choose 'Schematics L' from the 'Open with' drop down menu. OK the form.



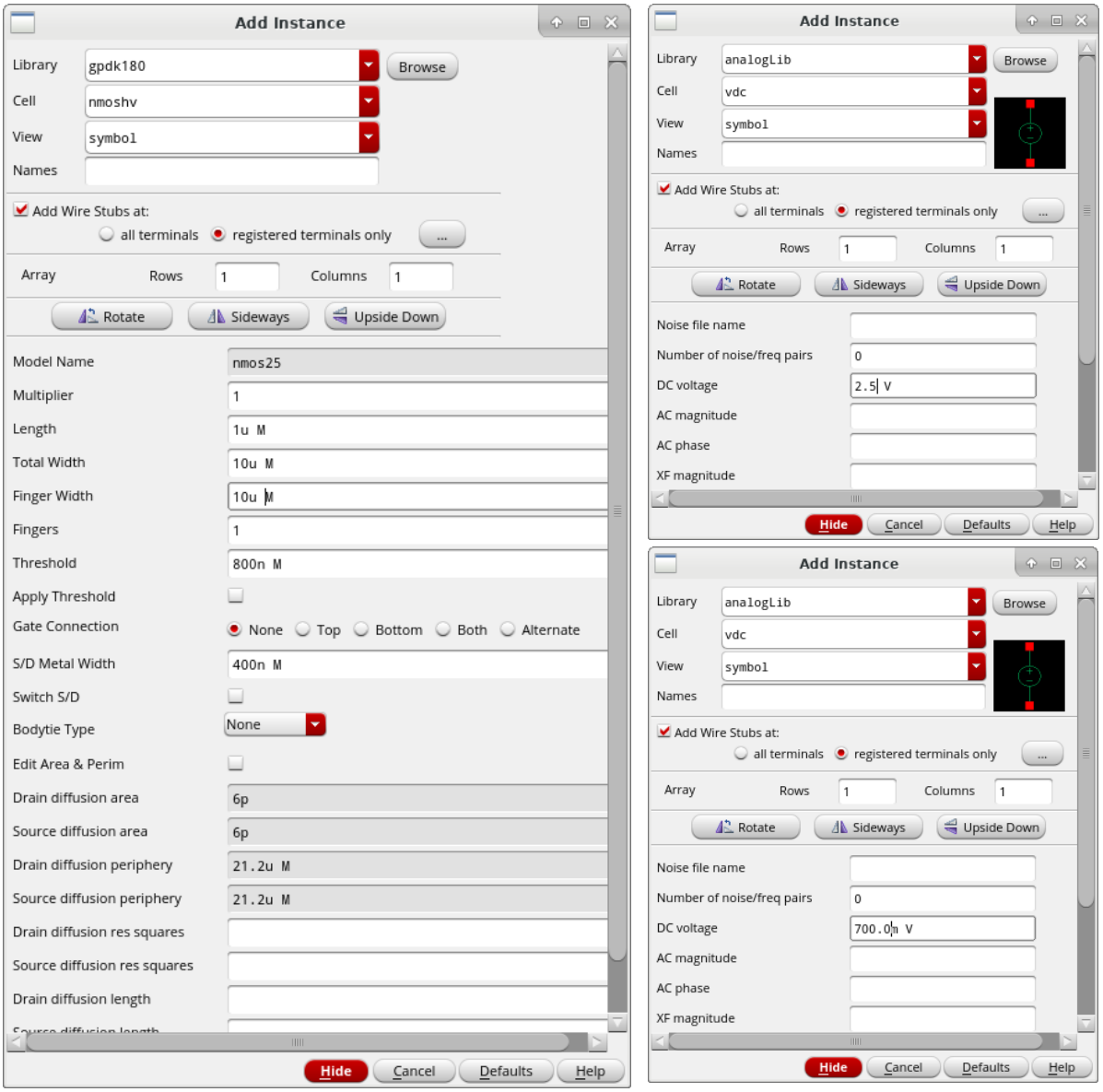
< Figure 31. The New File Form >

(4.1.3) You should be presented with a blank **Virtuoso Schematic Editor L** schematic entry window for the new cellview. This cellview will not be saved in the Library Manager until you check and save it in the Schematic Editor. From the Schematic Editor window, choose **Menu: File à Check and Save**. (Or you can also **Menu: Check à Current_Cellview** followed by **Menu: File à Save** to perform the same command.)

(4.1.4) In your schematic editor window, construct the circuit given in Figure 1. Instantiate and place two DC voltage sources (**vdc**) from the analogLib library, a ground connection (**gnd**) from the analogLib library, and one high-voltage NMOS FET (**nmoshv**) from the GPDK180 library. Set the DC supply voltage to 2.5V (**DC Voltage** = *2.5*), set the gate-to-source voltage to 0.7V (**DC Voltage** = *0.7*), and dimension the NMOS device 10u/1u (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

*Recall, to add a new part you can use **Menu: Create → Instance...**, the '**Create Instance**' icon-button on the toolbar at the top of the schematic window, or the keyboard shortcut '**I**'. You can also right-click anywhere in the empty space of the design canvas to invoke a context-menu and select the **Add Instance…** command from there. Refer to lab 12-1 for a refresher on how to add components to the schematic (sec. 4.2 Adding Components to the Schematic).*

*You can set the component properties at instantiation from the Add Instance form before placing them on the design canvas. Alternatively, you can modify the component properties after it has been placed on the design canvas, by first selecting the item and then by right-clicking on the item and selecting 'Properties…' You can also use **Menu: Edit → Properties → Objects**, the '**Edit Properties**' icon-button from the toolbar, or the keyboard shortcut '**Q**'. You can then modify the required attribute(s) and Apply/OK the change.*



< Figure 32. The nmoshv (GPDK180) and vdc (analogLib) Add Instance Forms. >

(4.1.5) Note that there are low-voltage MOS devices and high-voltage MOS devices in the GPDK180 library. In these labs, we will only be using the **nmoshv** n-type MOSFET and the **pmoshv** p-type MOSFET models. These are four-terminal devices (gate, source, drain, and body/bulk). Note the 'hv' suffix stands for high voltage. This is relative! For our purposes, high voltage means a maximum V_{DS} of 2.5V. This is in contrast to low-voltage devices with a sub-1V V_{DS}. (The high voltage behaviour is achieved by using a thicker gate oxide level.) Both the **pmoshv** and **nmoshv** devices have a threshold voltage V_T of ~0.5V.

When adding the **nmoshv** or **pmoshv** from the GPDK180 library, there are five parameters under the designer's control that set their operation and characteristics: the **length**, **total width**, **finger width**, **number of fingers**, and **multiplier**. This is typical of most MOSFET process development kits (PDKs). We address these briefly below:

1. The **length** attribute specifies the channel length of the device. For the GPDK180 library, the minimum size of the channel (its feature size, technology node, or process node) is 180nm. You can specify any size above this value (to the nearest 5nm) but not below. In these labs, since we are not operating at particularly high-frequencies, we choose not to use the smallest node – we will always default to a value of 1um. This ensures the first-order design equations we used to describe MOSFET operation (for the most part) provide parity with the simulations.

2. The **total width** attribute specifies the channel width of the device. For the GPDK180 library, the width may be set in the range 400nm to 50um (to the nearest 5nm). The **length** (L) attribute and the **total width** (W) attribute together specifies the *effective* width-to-length (W/L) ratio of the device.

3. The **finger size** and **number of fingers**. When laying out a MOSFET with a high W/L ratio, the designer has two options regarding the **shape of the gate**. Rather than a single, wide polysilicon run forming the gate (i.e., a single 'finger'), the gate is distributed into several equally spaced parallel fingers. This approach allows for better performance at high-frequencies (due to the reduction in gate parasitics and IR drop along the length of the gate) and greater layout flexibility (e.g. compactness) when using very wide devices. The effective W/L ratio (total-width/length) stays the same (though some aspects of circuit performance will be affected).

   For the GPDK180 library, the designer specifiers the number of fingers and, either, the total-width or the finger-width. Whichever attribute is not specified will automatically adjust according to the relation:

$$finger\ width = \frac{total\ width}{number\ of\ fingers}$$

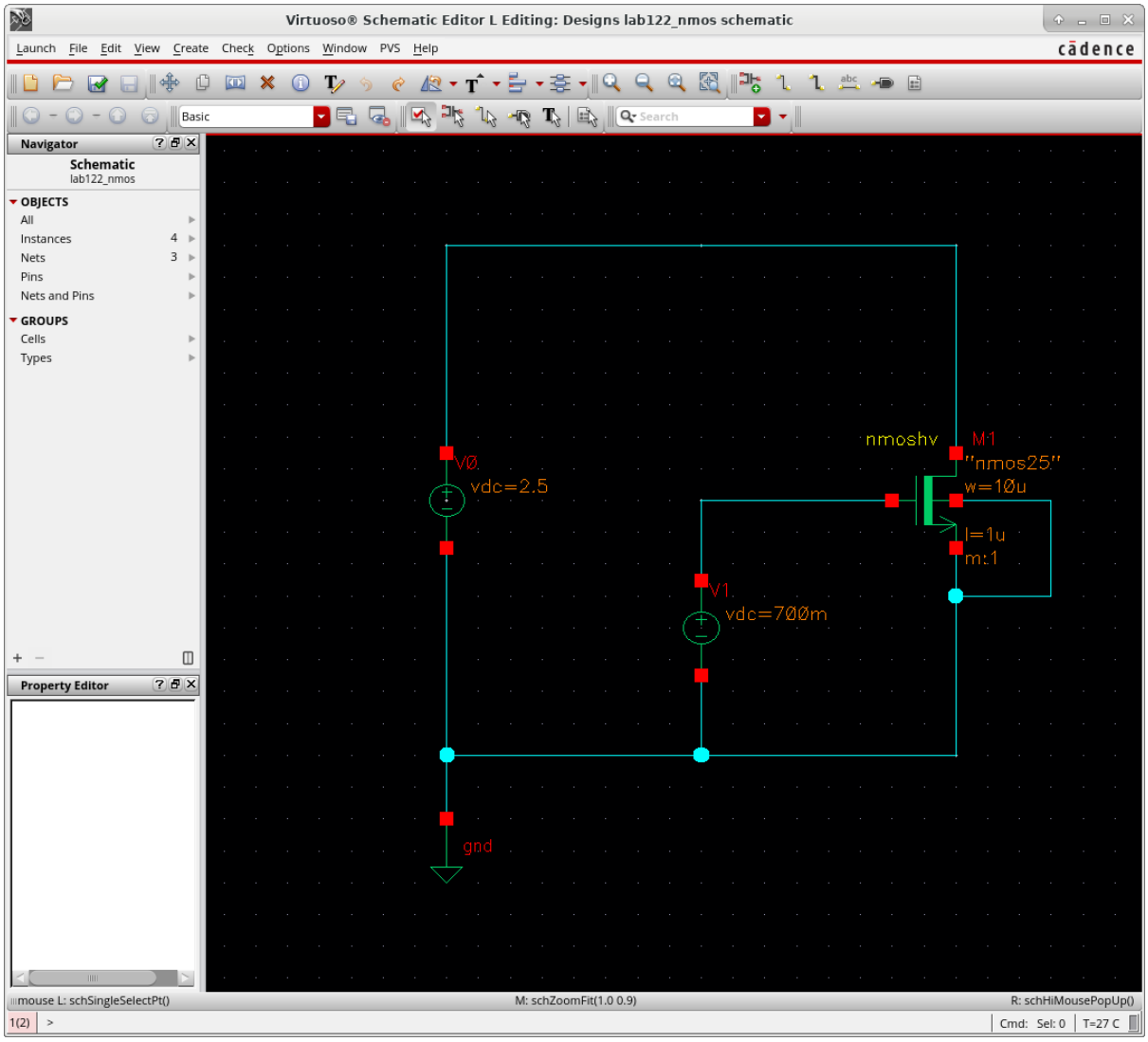
   In some PDKs, fingers are referred to as **stripes.** We will only use single fingers/stripes in this course.

4. The **multiplier** attribute allows several identical devices to be connected in parallel. This is used when the W/L ratio is high. For example, a device with a 50u/1u width/length ratio could also be constructed from five 10u/1u devices connected in parallel. In this case: length = 1um, total width = 10um, multiplier = 5. The multiplier is particularly useful for those cases where we want to scale current sources – for example in a current mirror. In such cases, better matching will be obtained by using multiple identical single-gate devices rather than one large device. We will make extensive use of the multiplier in the coming labs.

(4.1.6) With the components placed, you can now add wires to connect them together. You can use **Menu: Create → Wire (narrow)**, the '**Create Narrow Wire**' icon-button on the toolbar, or the keyboard shortcut '**W**' to start the wire command (see Figure 4).

*Recall that you can drag, move or stretch the components and wires after they have been placed on the design canvas. Left-click-and-hold the mouse button and drag the component or wire, or select the component and use the **stretch** or **move** commands. You can also rotate and flip the components before placing them using the **rotate**, **sideways**, and **upside-down** buttons on the add instance form. Or you can rotate/flip the components after placing them by selecting the component and using the **rotate** or **flip** commands, or by using **F3** with the **move/stretch** commands.*

(4.1.7) Check and save your schematic using the **Check and Save** command! Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.



< Figure 33. The wired NMOS testbench circuit in the schematic editor. >

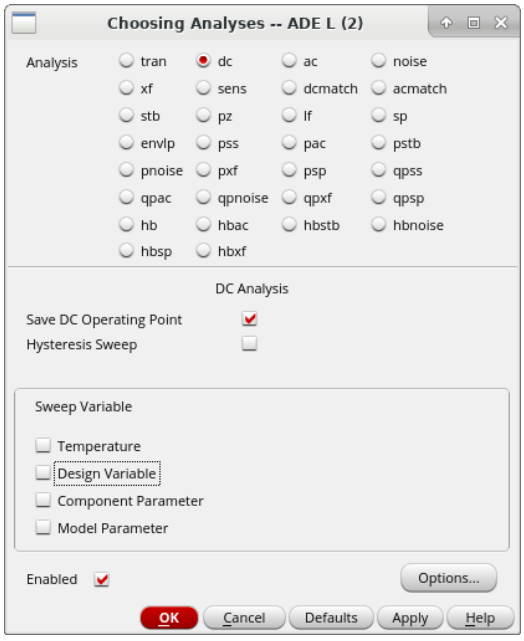
(4.1.8) NOTE: FOR AN NMOS DEVICE, YOU MUST ALWAYS CONNECT THE BODY/BULK TERMINAL TO THE MOST NEGATIVE NET (WIRE) OF THE CIRCUIT. IN FIGURE 4, THIS IS THE GROUND TERMINAL. FOR A PMOS DEVICE (NOT SHOWN) THE BODY/BULK TERMINAL MUST ALWAYS BE CONNECTED TO THE MOST POSITIVE NET (WIRE) OF THE CIRCUIT, USUALLY THE SUPPLY VDD. THIS IS NECESSARY TO ENSURE THAT THE CHANNEL WILL BE PROPERLY ESTABLISHED. THERE IS AN EXCEPTION TO THIS THAT WE WILL COME TO LATER.

4.2 Simulating the NMOS FET DC operating Point

(4.2.1) You should now have the completed schematic of the NMOS testbench circuit as shown in Figure 4. It should check and save without any errors or warnings.

(4.2.2) We will simulate the NMOS testbench using a DC Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch à ADE L**.

(4.2.3) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. On the form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. OK the form.



< Figure 34. The Choosing Analyses Form. Only DC Operating Point selected. >

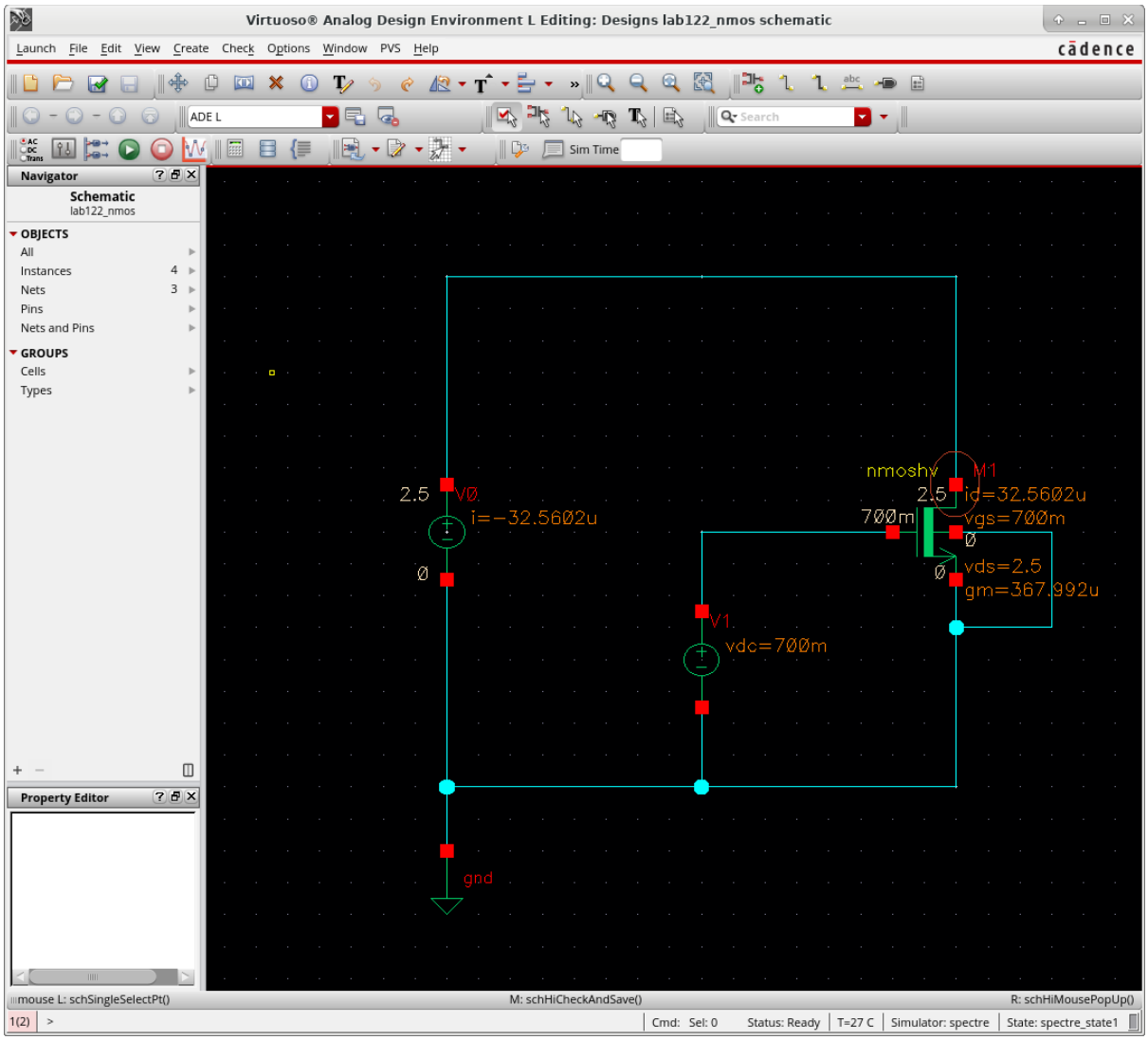
The DC operating point calculates the various node voltages and currents around the circuit, as well as various NMOS device parameters such as gm, IDS, VGS, VDS and more. The DC operating point will be calculated from the DC supply voltage (2.5V) and the NMOS gate-to-source voltage (0.7V).

(4.2.4) At this stage we are only simulating for the DC operating point. Simulating the DC operating point does not produce an output waveform, so we do not need to select any outputs to be plotted.

(4.2.5) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.

(4.2.6) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating point information annotated on the schematic including the DC bias currents, NMOS device transconductance gm, IDS, VGS, and VDS. See Figure 5.

Note: You can also turn annotations on and off by right-clicking on the canvas in the schematic editor window and selecting **Right-Click: Annotations…** There are number of annotations to choose from.



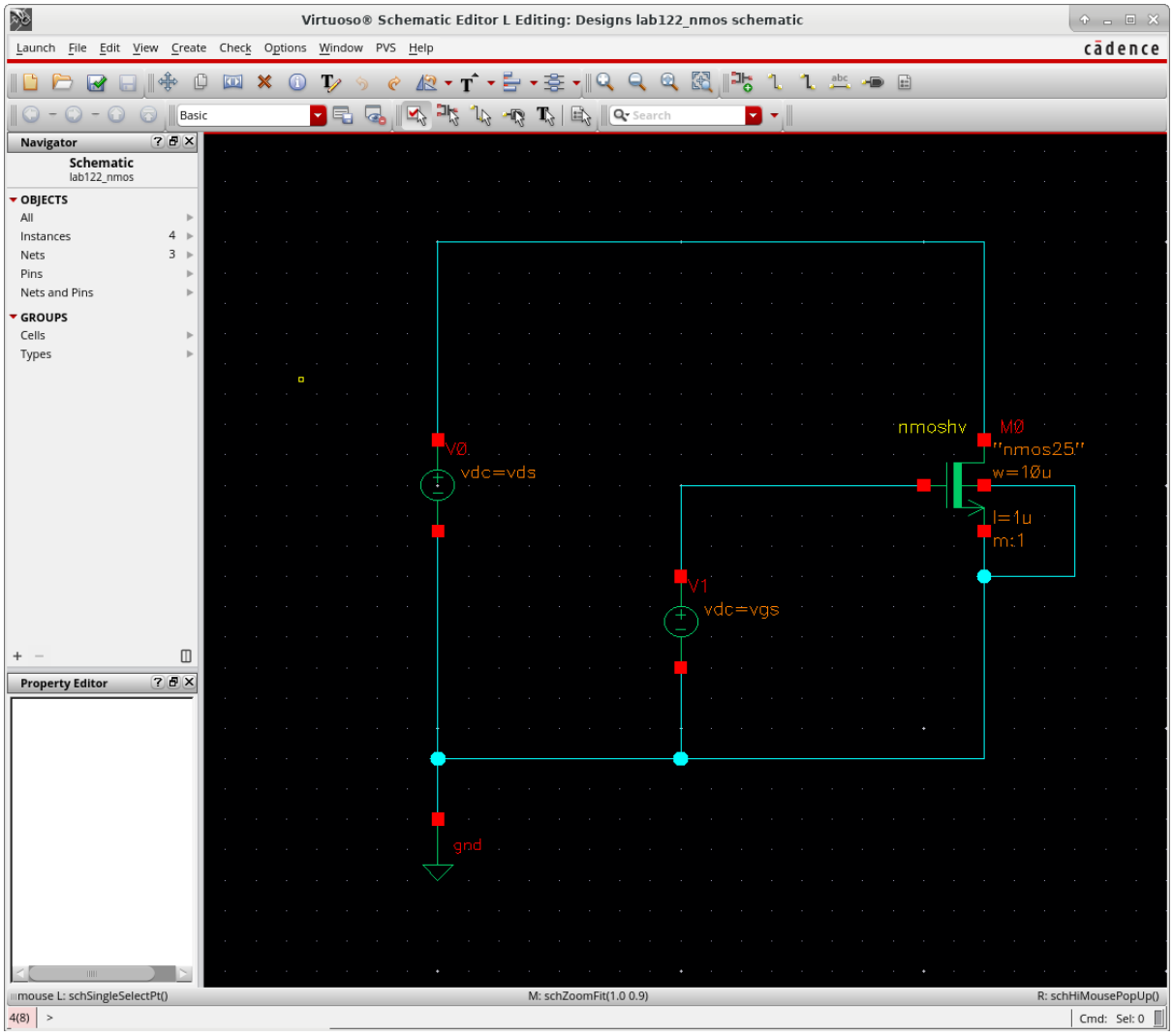
< Figure 35. DC Analysis – Save DC Operating Point. The wired NMOS testbench circuit with annotated DC node voltages and DC operating points. >

4.3 Parameterising the NMOS FET for a Swept-DC Analysis of V_{GS} vs. I_{DS}

(4.3.1) We will now simulate the testbench circuit using a swept-variable DC Analysis, aka a DC-sweep. In order to do this, we first need to modify the circuit slightly. Instead of giving each voltage source a fixed value, we will specify a parameter variable (aka a design variable).

(4.3.2) In the schematic window, modify both the DC voltage sources (**vdc**). For the DC supply voltage, change the 2.5 V value to the string vds (**DC Voltage** = *vds*) and for the gate-to-source voltage change the 0.7 V value to the string vgs (**DC Voltage** = *vgs*). This gives us two design variables – vdc=vds and vdc=vgs. **Check and save** your schematic after any modifications.

Note: you may need to annotate the component parameters so you can see the design variable names. From the ADE, choose **Menu: Results → Annotate → Component Parameters…** or from the schematic editor window, choose **Right-Click: Annotations → Component Parameters**.



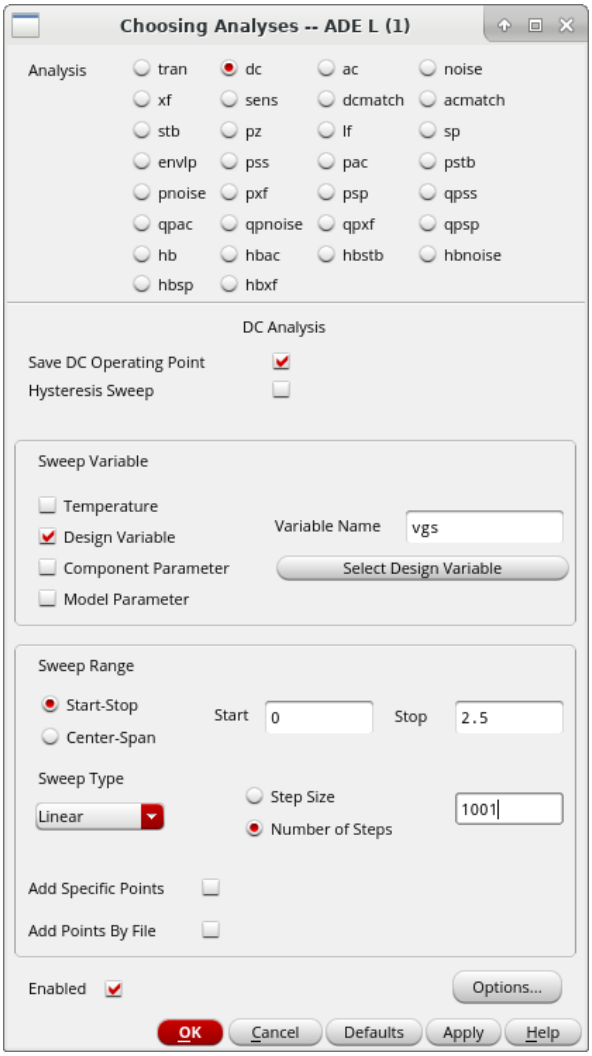
< Figure 36. The wired NMOS testbench circuit with dDesign Variables for the DC supply voltage (vdc=vds) and the DC gate-to-source voltage (vdc=vgs). >

In Figure 7, note that the DC supply voltage and DC gate-to-source voltage are now set using a design variable – vds and vgs respectively. During a simulation, the design variable(s) can be incrementally changed (according to a specified rule) and the simulation run for each and every increment. This allows a circuit to be simulated (swept) over a range of parameter values – and not at just one fixed bias/operating point.

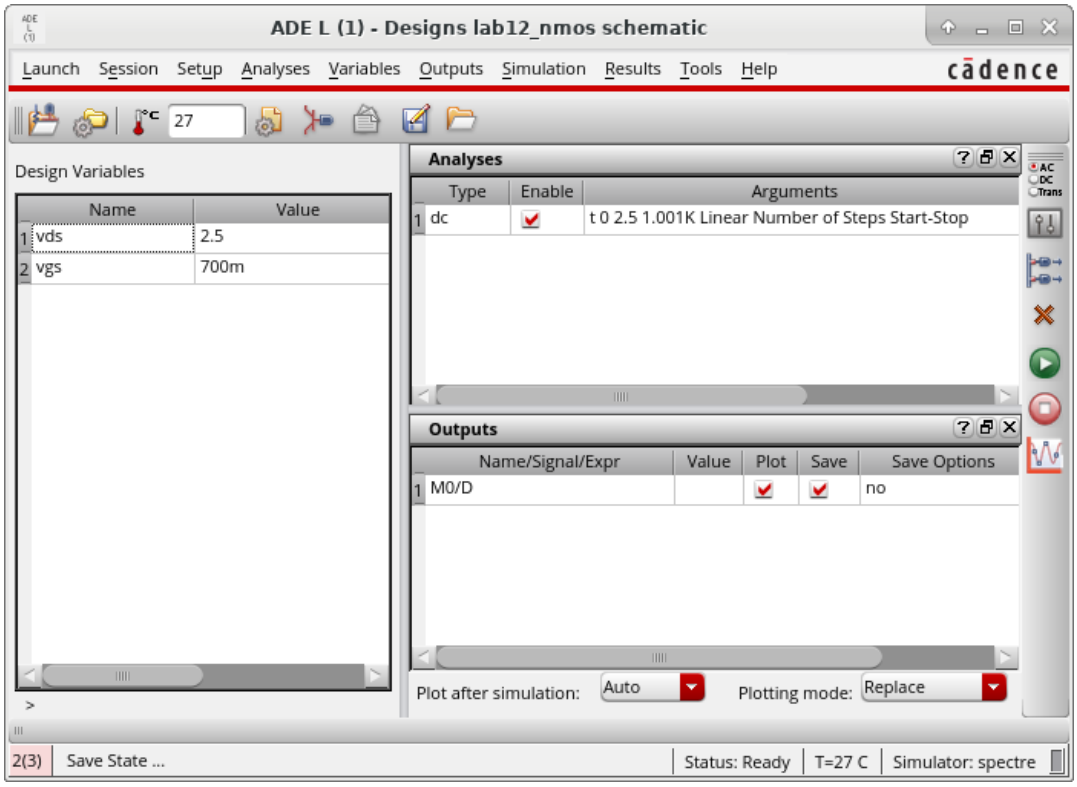
(4.3.3) Copy the design variables from the schematic into the ADE. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the *vds* and *vgs* variables in the 'Design Variables' list of the ADE. Initialise *vds = 2.5* and *vgs = 0.7*. These values are only used to calculate the DC operating point of the circuit.

(4.3.4) From the ADE window, edit the DC analysis (**dc**) by double clicking on it, or right-clicking on it and choosing edit from the context menu. We wish to sweep the input voltage (vgs) of the NMOS device from 0V to 2.5V while observing the drain current (IDS). We will use 1001 steps in the sweep to give the output waveform sufficient resolution.

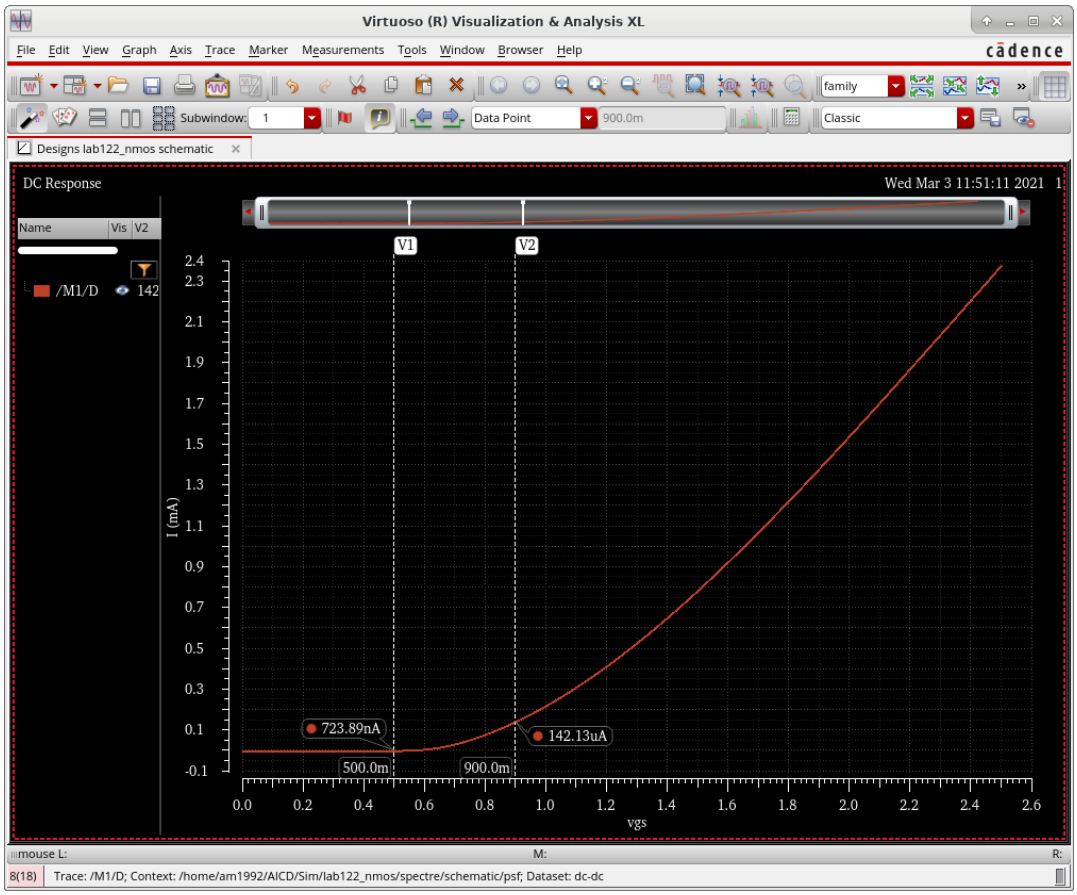
In the choosing analyses form. Check the ‘Save DC Operating Point’ box. Check the ’Design Variable’ box as the sweep variable. Type *vgs* as the variable name. Choose the ‘Start-Stop’ option button with Start = 0 and Stop = 2.5 for the sweep range. Select ‘Linear’ from the drop down menu as the sweep type. Choose the ‘Number of Steps’ option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the ‘Enabled’ box near the bottom of the form. See Figure 8. OK the form. The **dc** analysis should now appear in the ‘Analyses’ list of the ADE window.



< Figure 37. The Choosing Analyses Form for a DC sweep. >



< Figure 38. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 39. The waveform output of IDS vs. VGS with vertical markers at 0.5V and 0.9V. >
(4.3.5) Select the drain node current (IDS) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the drain node of the NMOS device. **Press the 'ESC' key to exit the plotting mode of the ADE**.

*Recall**,** if you want to plot a particular voltage, you click on the net/wire. If you want to save and plot the currents, you have to click a terminal **node** of a component (not the net/wire).*

(4.3.6) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error and both the text output window as well as the waveform output window should open automatically. You should get a plot of V_{GS} vs. I_{DS} – see Figure 10.

(4.3.7) Note: You can add markers to your graph. From the waveform output window, choose **Menu: Markers à Create Marker…** to bring up the 'Create Graph Marker' form.

From the marker form, you can choose the marker type (**point**, **horizontal**, or **vertical**). You can specify the precise location of the markers at instantiation, otherwise they will initialise/ appear at the origin and can be dragged to the wanted trace position. Double-clicking on a marker brings up the marker properties form, where you can refine its behaviour (you can also use **Menu: Marker à properties** to bring up the marker properties form – select the marker first.)

Alternatively, you can use the keyboard shortcuts to add a marker to your plot: '**m'** (point marker), '**h'** (horizontal marker), and '**v'** (vertical marker). When using the keyboard shortcuts, the marker will appear at the location of you mouse cursor.

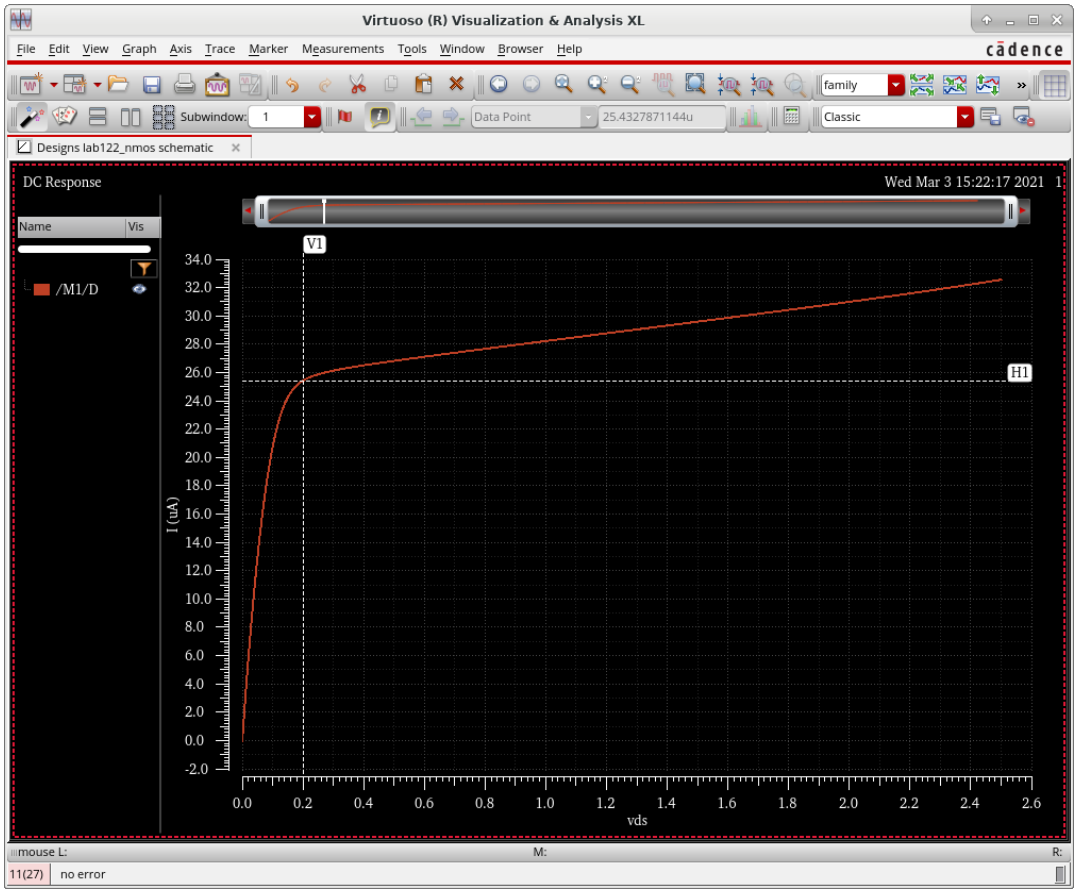
4.4 Parametric Swept-DC Analysis of V_{DS} vs. I_{DS} vs. V_{GS}

(4.4.1) We will now plot a DC sweep of V_{DS} vs. I_{DS} (for nominal V_{GS} = 0.7V). From the ADE window, edit the DC analysis (**dc**) to make $\underline{V_{DS}}$ the sweep variable. Either double click on the analysis or right-click on the analysis and choose 'Edit' from the context menu. Change the sweep variable name from V_{GS} to V_{DS}. Leave the rest of the form unchanged. OK the form.

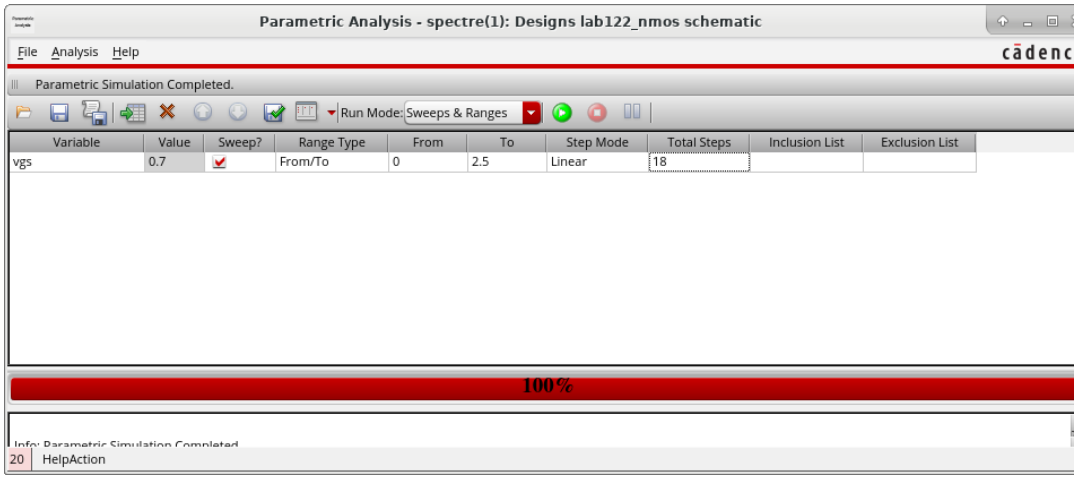
(4.4.2) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 11.

(4.4.3) Ultimately, we want to plot the V_{DS} vs. I_{DS} family of curves for increasing values of V_{GS}. (V_{DS} vs. I_{DS} vs. V_{GS}) This requires two sweeps – for V_{DS} from 0V to 2.5V and for V_{GS} from 0V to 2.5V. The DC analysis form only allows you to sweep one variable at a time, so we will need

to use the Parametric Analysis tool. From the ADE window, choose **Menu: Tools → Parametric Analysis…** to invoke the Parametric Analysis tool. See Figure 12.



< Figure 40. Single DC-sweep of IDS vs. VDS (VGS = 0.7V, VOV = ~0.2V, W/L = 10um/1um) >

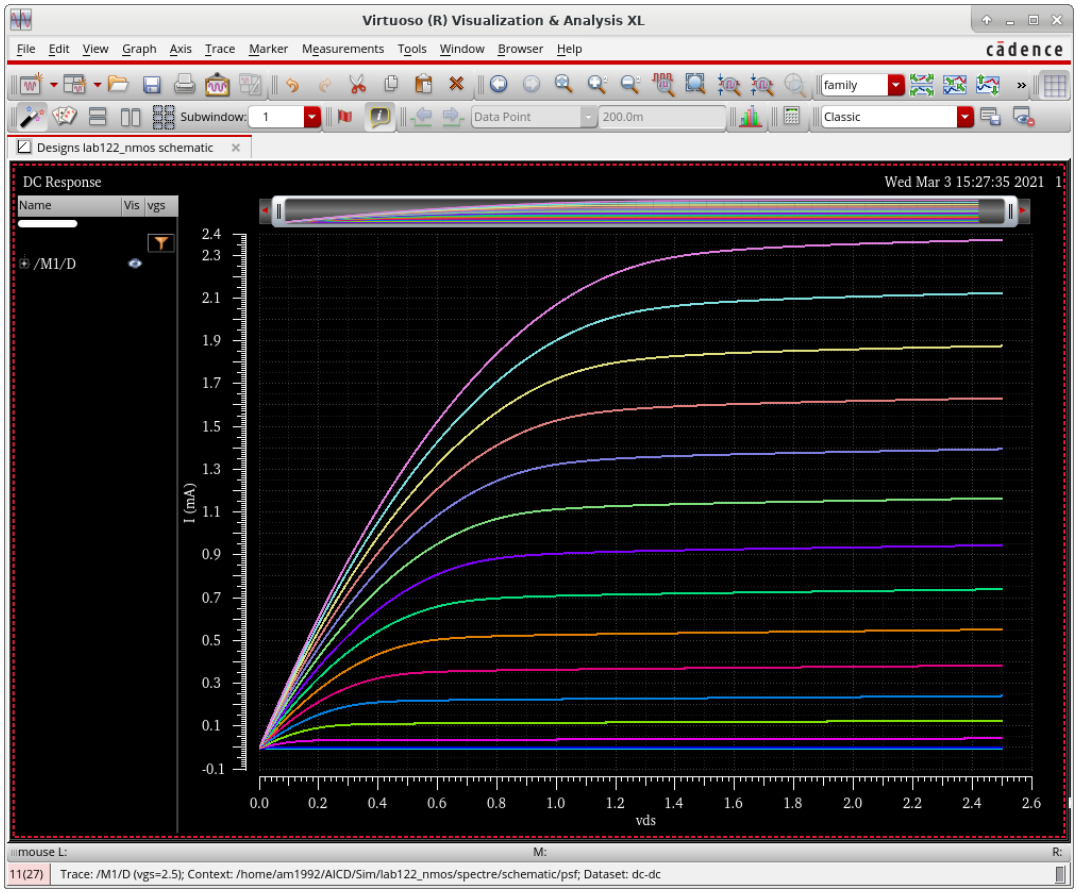


< Figure 41. Parametric Analysis Tool. Set up for a linear sweep of VGS → 0V to 2.5V in 18 steps. This will run the swept DC-analysis (dc) 18 times to create a family of curves. >

Note: The parametric tool works by repeating an analysis/simulation a specified number of times. While the parametric analysis tool allows us to specify additional variables to sweep, it is typically used to vary a parameter using far fewer sweep points than a DC-sweep, as it is much slower. The parametric analysis will run an analysis in the Analyses section of the ADE, once for each step of a parameter variation. A swept DC-analysis can efficiently handle sweeps of tens of thousands of sweep points. For the parametric tool, sweeps are best kept below 100 sweep points. A parametric sweep can be specified as a range to sweep or as an explicit list/set of values over which to iterate. (Run Mode: Sweeps and Ranges, or Parametric Set).

(4.4.4) In the Parametric Analysis window, select Sweeps and Ranges from the '**Run Mode'** drop down menu in the tool bar. In the '**Variable'** column, you should get a drop-down menu of the available design variables. Select V_{GS} as the parametric sweep variable. Note, only the design variables in the 'Design Variables' section of the ADE will be available in the drop-down list along with the system variable temp. So, make sure you have copied all design variables from the schematic into the ADE using the **Menu: Variables → Copy From Cellview…** command.

In the '**Range Type**' drop-down menu select 'From/To'. In the 'From' column type 0 as the start value of the parametric sweep, and in the 'To' column type 2.5 as the stop value of the parametric sweep. In the '**Step Mode**' column select 'linear' and in the **Total Steps** type '18'.



< Figure 42. Parametric DC-sweep of IDS vs. VDS for VGS à 0V to 2.5V. (18 steps, W/L = 10um/1um) >

(4.4.4) The parametric analysis is run from the parametric analysis window not from the ADE window. In the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 13. With the above settings, the ADE will perform a swept-DC analysis of V_{DS} from 0V to 2.5V, a total of 18 times – one simulation for each value of V_{GS} (V_{GS} will be linearly incremented from 0V to 2.5V with each increment of V_{GS} given by 0.134V (=2.5V/18) as specified in the parametric form).

(4.4.5) **Discussion**. The plots of Figure 10 (V_{GS} vs. I_{DS}), Figure 11 (V_{DS} vs. I_{DS}), and Figure 13 (V_{DS} vs. I_{DS} vs. V_{GS}), should not be new to you. Refer to section 1.2.4 of the background material – for a discussion on the large-signal terminal characteristics of the MOS device.

As pointed out in the introduction to this part of the lab, the point of the above exercises was to simulate the large-signal behaviour of the MOSFET device from which the small-signal model of the MOSFET *as an amplifier* is derived. Recall, the fundamental gain A_V of a MOSFET amplifier is given by,

$$A_V = g_m \times r_O || R_D$$

where g_m is the small-signal transconductance of the MOSFET, r_o is its inherent output resistance, and R_D is the load. The small-signal transconductance g_m and output impedance r_o relate to the large-signal characteristics we have just simulated by,

$$g_m(V_{GS}) = \frac{\partial I_{DS}}{\partial V_{GS}} \qquad r_o(V_{DS}) = \left[\frac{\partial I_{DS}}{\partial V_{DS}}\right]^{-1}$$

When using a MOSFET as an amplifier, we must first bias the device at an appropriate DC operating point – the fixed DC values of V_{GS}, I_{DS}, and V_{DS} – to maximise the performance of the device (to a given specification). This comes down to operating the MOSFET at a point along the curves of V_{DS} vs. I_{DS} and V_{GS} vs. I_{DS}, that maximise both g_m and r_o, while giving due consideration to other factors (often equally as important) such as linearity, signal-swing, frequency response, stability, and power consumption. We will develop the background required for making such a decision in the rest of this lab and the coming labs.

An amplifier comprises a transconductor and a load. In the next section, we will move from the characterisation of the MOSFET as a VCCS (the transconductor), to the basic MOSFET amplifier – by the addition of a load.

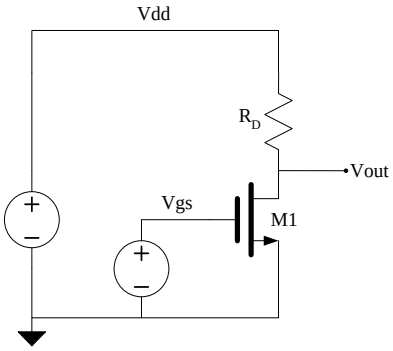
5. Circuit Simulation Exercises: Part 2

With guidance, you will:

- Create and simulate a simple resistor-loaded single-stage NMOS FET voltage amplifier.
- Address the inadequacies of using a resistive load for IC design. Create and simulate a current-source loaded (or active-loaded) NMOS FET voltage amplifier.

5.1 A Simple Voltage Amplifier

(5.1.1) Create the schematic for the basic resistor-loaded voltage amplifier circuit in Figure 14. A summary of the circuit's operation is given in section 4.2.1 of the background material.



< Figure 43. Basic Resistor-Loaded Voltage Amplifier
Reproduced from Figure 15, section 4.2.1 of the background material. >

(5.1.2) Note: Since you have already built most of this circuit in the first part of this lab, there are a number of ways you could approach creating this new schematic. You could:

1. Modify your existing schematic (saving it under the same name).
2. Save or create a copy of your existing schematic with a new name, and either continue to modify your existing schematic or switch to working on the new copy.
3. Create a new schematic from scratch.

Option (1) is quick and easy – but in doing so you lose the history or progression of your work. For example, if you wanted to replot the characterisation exercises you completed in the first part of this lab, you would have to rebuild the circuit as it was then, as you will have since modified the circuit. For simple circuits, with few modifications, this may not be much of a problem. For complex circuits, however, it is good practice to keep a stable working copy of your design at various stages of completion that you can revert to if things go wrong.

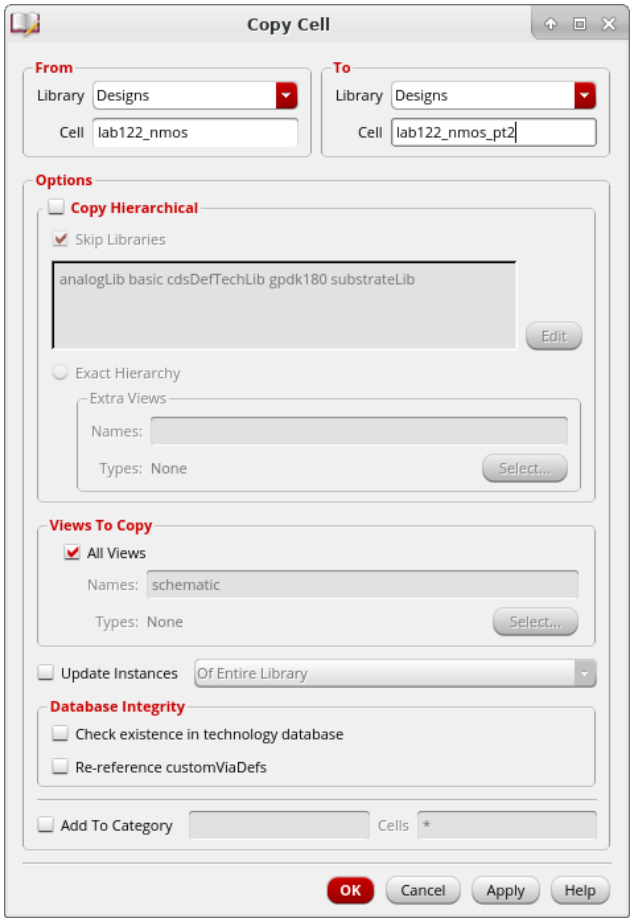
Option (2) is the most efficient way of starting a new design – by reusing as much of an existing design as possible while keeping the existing working design safe (unmodified). In this case,

give each copy of the schematic a meaningful name and keep the modified versions in one library to make any changes to the circuit easier to identify (assuming you are not using a version control system). In cadence, there are two ways of creating a new copy of an existing schematic – from the Library Manager or from within the Schematic Editor.

Option (3) is recommended as you are still learning Cadence. Since you are still learning simple theoretical concepts of the analogue IC design flow, and your circuits are simple, take the opportunity to remind yourself of how to perform the basic operations of the software (e.g., starting a new schematic and/or simulation from scratch) until it becomes second nature. This will ensure that when you come to learn the more difficult concepts in the analogue IC design flow you are not held back by your lack of familiarity with the software.

(5.1.3) To start a new schematic from scratch, use the New File form as you have done previously – ensuring you are in your Designs library – giving your new schematic a meaningful name such a 'lab122_nmos_pt2'.

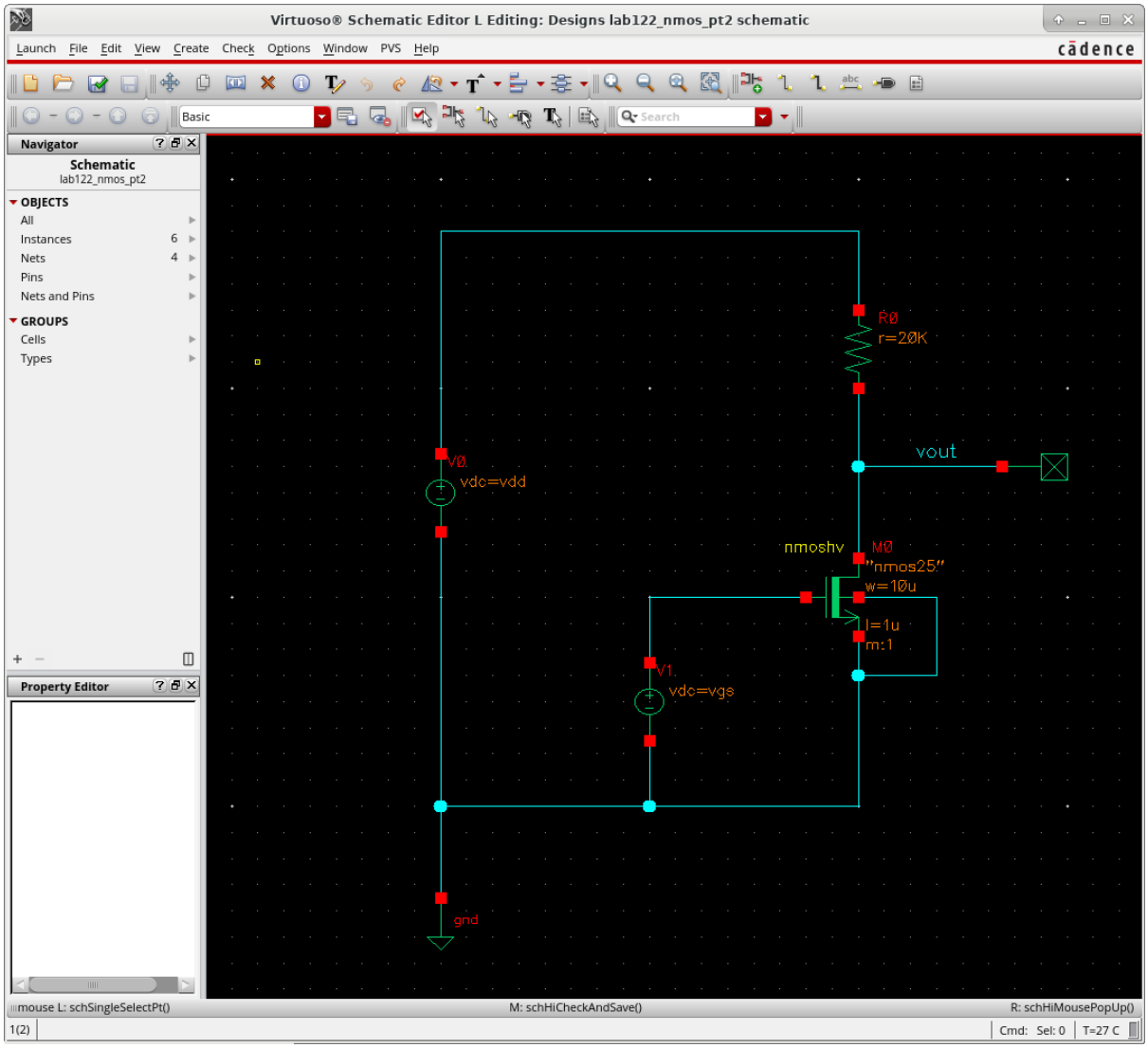
Alternatively, to create a copy from an existing schematic, go to the Library Manager window and select the cell you want to copy e.g. lab122_nmos from the Designs library. Right-click on the cell and select 'Copy…' from the context menu to invoke the 'Copy Cell' form. (You could also select **Menu: Edit à Copy** from the menu bar.)



< Figure 44: The Copy Cell Form >

In the ‘From’ section of the copy cell form, select the source library and cell name and in the ‘To’ section choose the destination library and cell name (in this lab, use the Designs library in both the From/To sections). Give the new cell a meaningful name e.g. ‘lab122_nmos_pt2’. Leave the rest of the form unchanged and OK the form. In the library manager, in the Designs library, you should now see a new schematic cellview called ‘lab122_nmos_pt2’ (or whatever name you chose).

You could also save a copy from the Schematic Editor window from **Menu: File à Save a Copy…**, specifying the library, cell name, and cell type. Note, that in this case when you save a copy you remain working in your original schematic – you do not automatically switch to the new copy. You should see the new copy in the Library Manager.



< Figure 45. The Resistor-Loaded Voltage Amplifier Schematic as in Figure 14>

(5.1.4) In your schematic editor window, create the circuit given in Figure 15. Instantiate two DC sources (**vdc**), a resistor (**res**), and an NMOS FET from the GPDK180 library (**nmoshv**). Parameterise the DC supply voltage as *vdd* (**DC Voltage** = *vdd*), parameterise the gate-to-source voltage as vgs (**DC Voltage** = *vgs*), set the load resistor to 20KΩ (**Resistance** = *20K*), and dimension the NMOS device 10u/1u (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

(5.1.6) Wire the components as in Figure 15 using the **Wire (narrow)** command. Name the output net *vout* using the **Create Wire Name** command. Use the **noConn** connector from the **basic** library to avoid floating-net warnings. Keep the schematic neat.

(5.1.7) Check and save you schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

5.2 Simulating the Voltage Amplifier using a Swept-DC Analysis

(5.2.1) You should now have the completed schematic of the resistor-loaded voltage amplifier circuit as shown in Figure 16. It should check and save without any errors or warnings.

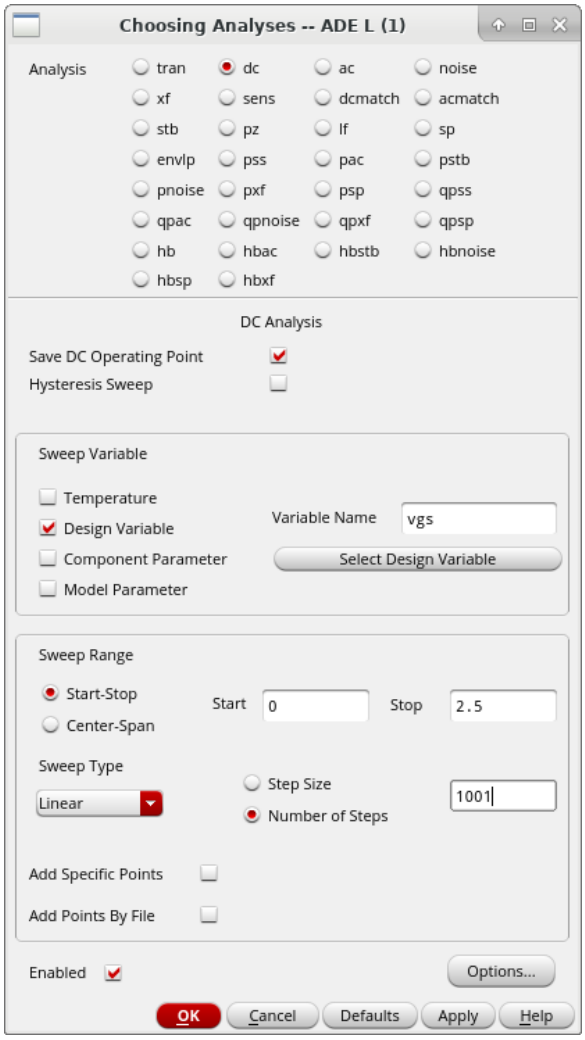
(5.2.2) We will simulate the basic voltage amplifier circuit using a swept-variable DC Analysis, aka a DC-sweep. From the Schematic Editor window, launch the Virtuoso Analogue Design Environment (ADE) window from **Menu: Launch à ADE L**.

(5.2.3) Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the two **vdc** components - *vdd* and *vgs*. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the *vdd* and *vgs* variables in the 'Design Variables' list of the ADE. Initialise *vdd = 2.5V* and *vgs = 0.7V*. These values are used to calculate the DC operating point of the amplifier circuit.

(5.2.4) From the ADE window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. From the form, select the '**dc**' option button.

(5.2.5) We wish to sweep the input voltage (vgs) of the NMOS device from 0V to 2.5V while observing the output voltage (vout). We will use 1001 steps in the sweep to give the output waveform sufficient resolution.

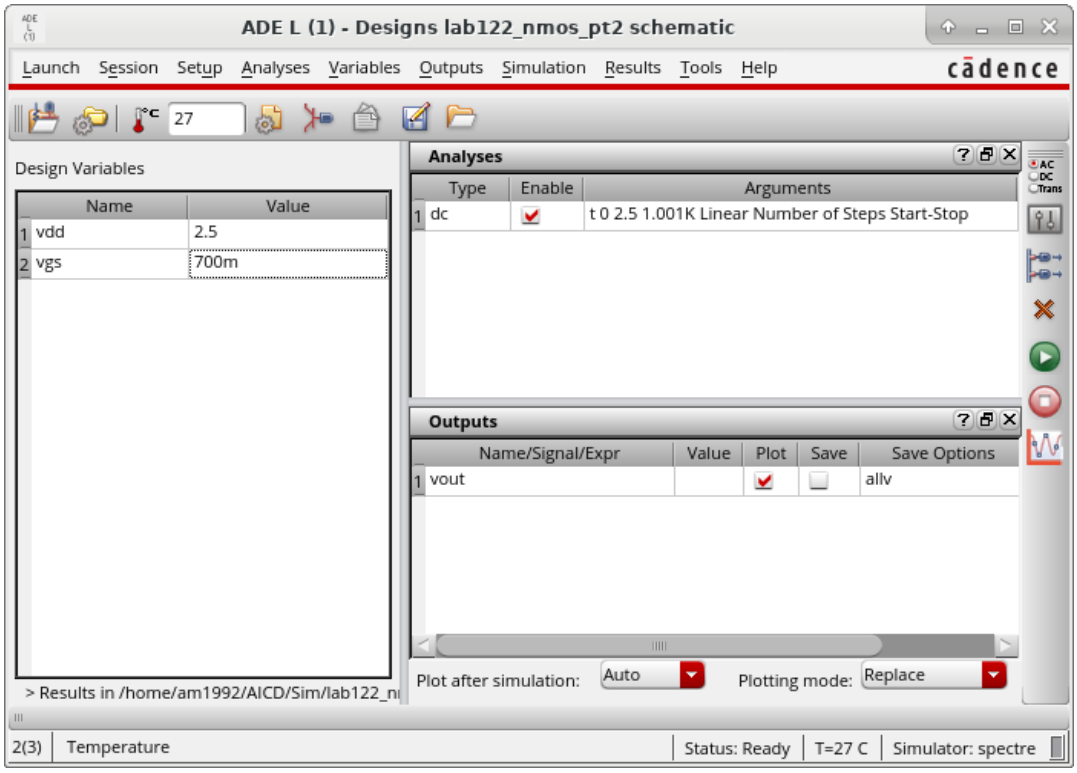
In the choosing analyses form. Check the ‘Save DC Operating Point’ box. Check the ’Design Variable’ box as the sweep variable. Type ‘vgs’ as the variable name. Choose the ‘Start-Stop’ option button with Start = 0 and Stop = 2.5 for the sweep range. Select ‘Linear’ from the drop-down menu as the sweep type. Choose the ‘Number of Steps’ option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the ‘Enabled’ box near the bottom of the form. See Figure 17. OK the form. The **dc** analysis should now appear in the ‘Analyses’ list of the ADE window.



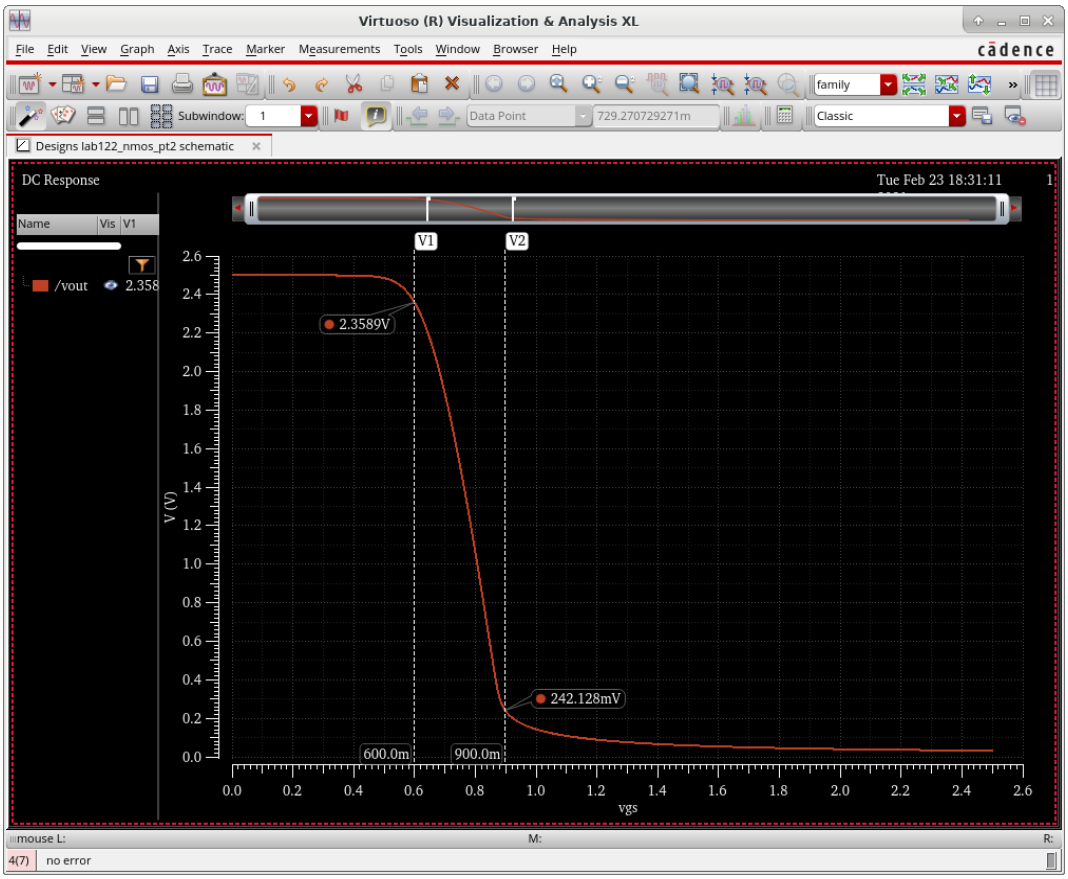
< Figure 46. The Choosing Analyses Form. >

(5.2.6) Select the output net voltage (vout) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the output net *vout*. **Press the ‘ESC’ key to exit the plotting mode of the ADE**.

(5.2.7) In the Analog Design Environment window, run the ‘**Netlist and Run**’ command by clicking on the green play button. If you followed all the instructions correctly, the simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 19.



< Figure 47. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 48. The waveform output window with vertical markers at 0.6V and 0.9V. >

5.3 Discussion: The Inadequacies of Using a Resistive-Load

The Voltage Transfer Curve

Both the PMOS and NMOS devices from the GPDK180 library that we will be using in these labs are high-voltage devices (**nmoshv** and **pmoshv**) and have a threshold voltage ~0.5V.

Observe the DC voltage transfer curve (VTC) of the basic voltage amplifier (Figure 19). With V_{GS} below the threshold voltage, the NMOS device is cut-off. As no current flows through the resistor, there is no voltage drop across it, and so V_{OUT} sits at the 2.5V supply voltage. As V_{GS} approaches 0.5V, the threshold voltage, we begin to turn on the device. And, as V_{GS} pushes over and above 0.5V, very readily we get some transfer from the power-rail (2.5V) at the top down to the ground-rail (0V) at the bottom. Notice that everything goes on in between the first 0.3V of the overdrive voltage ($0V \leq V_{OV} \leq 0.34V$) – that is, between 0.6V and 0.9V. Not a lot happens before or after that.

Given the DC voltage transfer curve – we pose the question: where on the transfer-curve do we bias or operate the device? For analogue design we want to operate in the middle of the sweep/transition, at $V_{OUT} = V_{DD}/2$, as this is where we can support an analogue signal transfer through the system with maximum swing (positive and negative excursions about the output operating point). Given that the transfer curve is somewhat non-linear, we may adjust the above statement to: we want to operate the device in the middle of the most linear part of the sweep/transition. We, therefore, want to bias the device at the input such that, for the nominal input voltage, the output sits close to mid-rail where we'll get the maximum, linear voltage swing. For the VTC in Figure 19, we would set the input DC bias at about 0.7V to 0.8V.

(As an aside, if we were interested in digital electronics we would only be interested in the extremes – 2.5V and 0V, where the transistor is wholly-on or (very nearly) wholly-off – since it's a binary system. In fact, the original logic gates were implemented by precisely this type of circuit – so-called Resistor-Transistor Logic. Nowadays, of course, CMOS based digital logic has become mainstream. You will have seen the CMOS previously on the programme.

Note that the output does not fully go to 0V. We've got a complete mapping to the 2.5V – this is simply because when we are below (or significantly below) the threshold voltage, we are not passing any current though the NMOS device or through the resistor, and with no voltage drop across the resistor the output is at the supply-rail.

However, we can't quite reach zero because there is always some residual resistance of the device. That is, at the lower end of the transfer curve, we have pushed the device out of the saturation region ($V_{DS} \geq V_{GS} - V_T$), into the triode region ($V_{DS} < V_{GS} - V_T$), and into the linear region ($V_{DS} << V_{GS} - V_T$) where we've effectively got some resistance. This leads to a voltage drop across the device that prevents the output going fully to 0V rail. If the output were to go

fully to the 0V rail, we would have a V_{GS} but no V_{DS}, and we would have effectively turned off the device (V_{DS} = I_{DS} = 0).
Note: If we were to use a PMOS device as the transductor and have the resistor as the load connected to the 0V rail – the opposite would be true – we could reach the 0V rail but not the 2.5V rail.

Limitations of the Resistive Load

From the VTC, we have a voltage gain of around $\Delta V_{OUT}/\Delta V_{IN}$ = 1V/0.1V = 10 (although not perfectly linear across the sweep). Note, that this is not how we normally measure the small-signal gain of a circuit – we will use an AC analysis for that in the next lab – but we should be able to extract, graphically, the DC operating point information (chiefly *gm* and r_o) from the DC-sweep simulation.

There is, however, a problem with the resistor-loaded architecture. We know the relationship between current, voltage and resistance. If we want to make the transfer curve steeper – that is, increase the voltage gain – we need a larger load resistance (assuming we have a fixed transconductance). But, if we were to increase the load resistance, while keeping the same drain current, we upset (lower) the DC operating point at V_{OUT}—lowering the available signal swing. We can, in turn, reduce the I_{DS} bias current. However, for large gains we require large resistive loads, this may require a very small drain current and therefore a very small V_{GS}. As we decrease V_{GS} we come out of the predictable square-law region of MOSFET operation, where all the design equations/approximations work, and enter the exponential region. In this region, the paper design equations no longer work, device models are no longer accurate, and we operate awkwardly close to the threshold voltage – where process, voltage, and/or temperature (PVT) variations may accidentally turn the MOS device off completely. In addition, the exponential region of operation is noisy due to the low drain current density.

So, whilst we can increase the gain by increasing the resistance, this leads to many problems with respect to the DC operating point. (The DC bias point at V_{OUT} will drop and we are no-longer at mid-rail. We now have to reduce the drain current to bring that mid-rail voltage up, but we've got a problem with pushing less current through the device – operating in the exponential region / close to the threshold voltage, susceptibility to PVT, questionable model accuracy, and device noise.) Alternatively, we could increase the transconductance to increase gain. But this means drawing more DC bias current – lowering overall power efficiency. Increasing the drain current also decreases r_o, which appears in parallel with the load resistor. As a result, the increase in gain is limited.

The Resistor-loaded voltage amplifier, while simple, is beset with problems for IC design giving a relatively low gain and a lack of configurability of gain. So, what would make an ideal load? ***We need to decouple the setting of the gain, from the setting of the DC bias/operating point. We want a very high impedance load (ideally infinite), while being able to set the drain-bias current arbitrarily. We do not want the DC voltage across the load to be a function of the load current.***

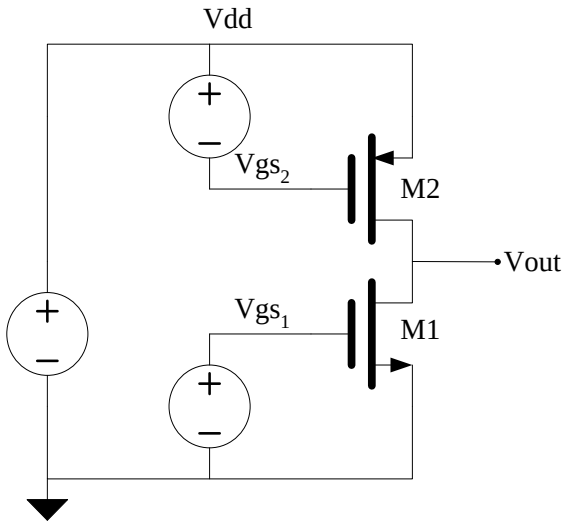
The ***ideal current-source*** has an infinite small-signal (incremental) impedance. Yet, the DC voltage across it is not dependent on the current through the source. If the load of the basic voltage amplifier was an ideal current source, it would define the current through the device while presenting an (ideally) infinite impedance. This would give us a very high gain. Note that despite the infinite impedance of the load, the gain would not be infinite since it appears in parallel with the r_o of the transconductance device (NMOS) which is finite.

Recall that the MOSFET device is a transconductance device. It therefore behaves as a voltage controlled current source (VCCS). A constant-current source can thus be implemented from a MOSFET device with a constant gate-to-source bias. In the circuit of Figure 14, we can use a PMOS device to implement a current-source load. Together, the NMOS transconductor and PMOS load, form the basic building-block of the IC voltage amplifier – the CMOS gain cell. We will explore this in the next section.

5.4 Replacing the Resistor-Load with a Current-Source Load

(5.4.1) In this part of the lab, we will implement a current-source loaded single-stage NMOS FET voltage amplifier to redress the inadequacies of using a resistive-load.

(5.4.2) Modify or copy your existing voltage amplifier schematic, or create a new schematic for the current-source loaded voltage amplifier circuit. You should now be able to create, edit, and modify a schematic with minimal direction.



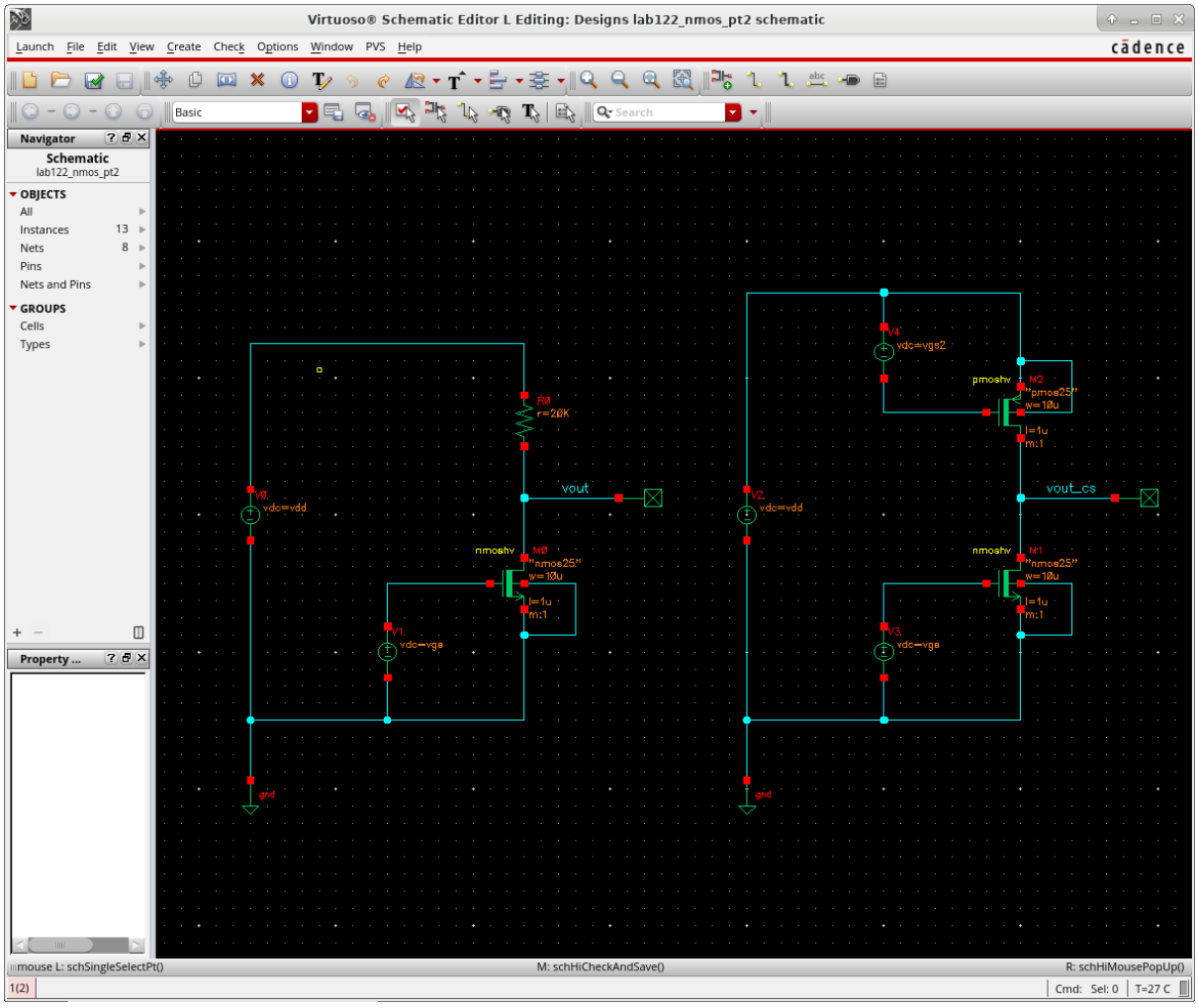
< Figure 49. Current-Source Loaded Voltage Amplifier
(Reproduced from Figure 16, section 4.2.2 of the background notes). >

(5.4.3) In this case, we would suggest building the new current-source loaded amplifier circuit alongside your existing resistor loaded amplifier circuit so you can compare the performance of the two circuits side-by-side (two circuits in one schematic). See Figure 21.

(5.4.4) Copying the circuit. Since the circuits of Figures 15 and 16 are similar (except for the load device), if you want to create the new circuit alongside your previous circuit, you can save time/effort by duplicating part or all of the previous circuit.

In the schematic window, on an empty part of the design canvas, left-click-and-hold the mouse button and drag a selection box around the circuit you want to duplicate. When you release the mouse button, the selected circuit will be highlighted pink/purple. Choose the **Copy** command (**Menu: Edit → Copy** or press '**c**') and click on a highlighted part of the circuit. A copy of the circuit should now attach itself to and move with the cursor. You can instantiate the copy by left-clicking anywhere on the design canvas.

(5.4.5) Modify you second circuit by replacing the resistor-load with a PMOS FET device from the GPDK180 library (**pmoshv**). Add a dc voltage source (**vdc**) for the PMOS gate-to-source bias. We will keep all other component properties the same as they were for the resistor-loaded amplifier. (Note the orientation of the vdc component providing vgs for the PMOS device – for a PMOS device, the gate must be negative with respect to the source.)



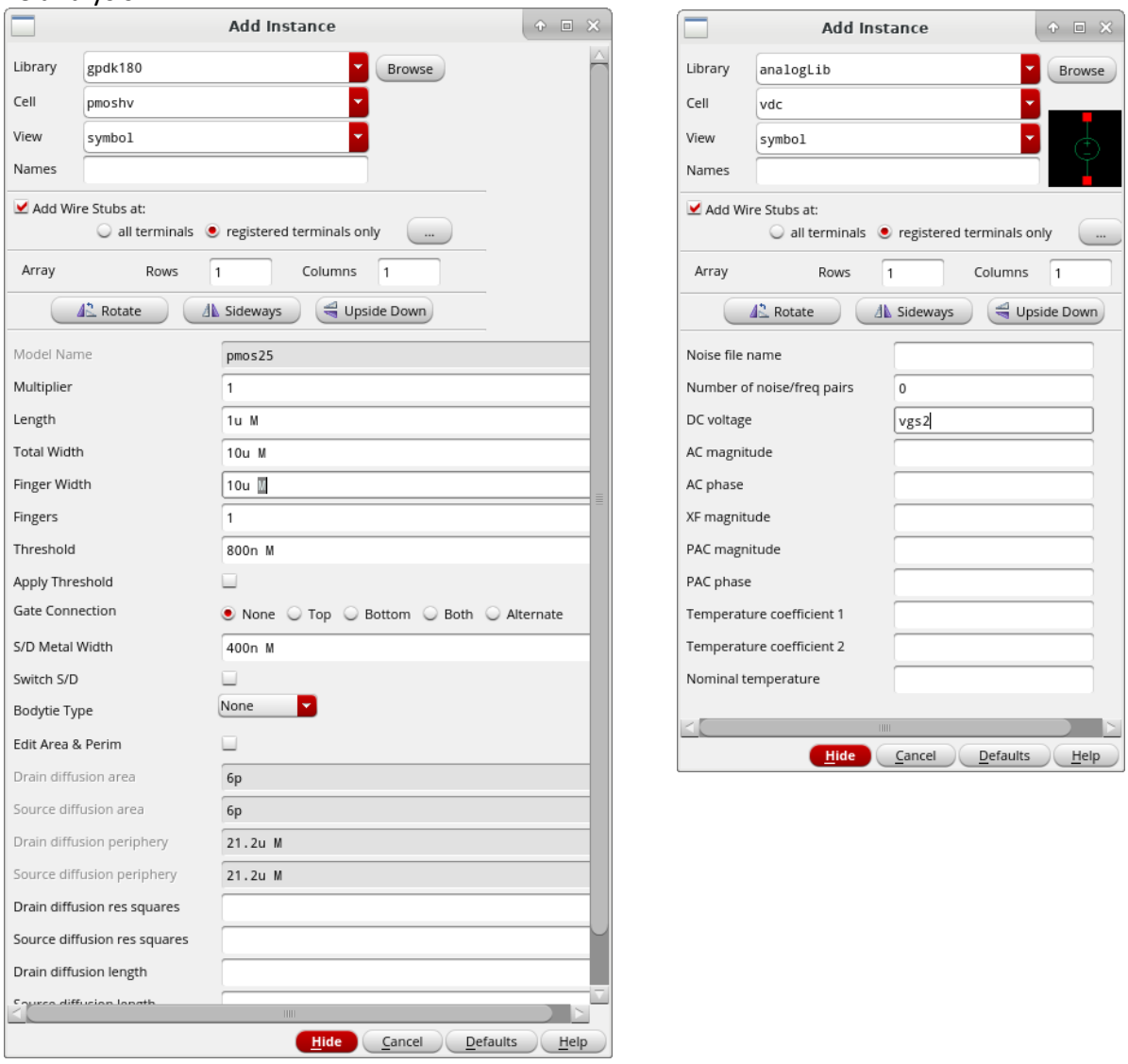
< Figure 50. The completed Current-Source Loaded Voltage Amplifier schematic alongside the original Resistor-Loaded Voltage Amplifier>

(5.4.6) Give the PMOS load device a W/L ratio of 10u/1u ('**Total Width**' = *10u*, '**Length**' = *1u*). Parameterise the PMOS gate-to-source voltage as 'vgs2' (**'DC Voltage'** = *vgs2*). Leave all other attributes/properties unchanged.

(5.4.7) Name the output net of the current-source loaded amplifier circuit '*vout_cs*' to distinguish it from the output net of the resistor loaded amplifier *vout*. Note: When you name the output net of the new circuit, it must not have the same name as the output net of your previous circuit (vout). Giving two separate wires or nets the same name will create a logical connection between those wires/nets – they will become one shared net. This behaviour can be useful to avoid excess wiring in your schematic reducing clarity, but is not wanted here.

(5.4.8) Check and save your schematic using the **Check and Save** command.

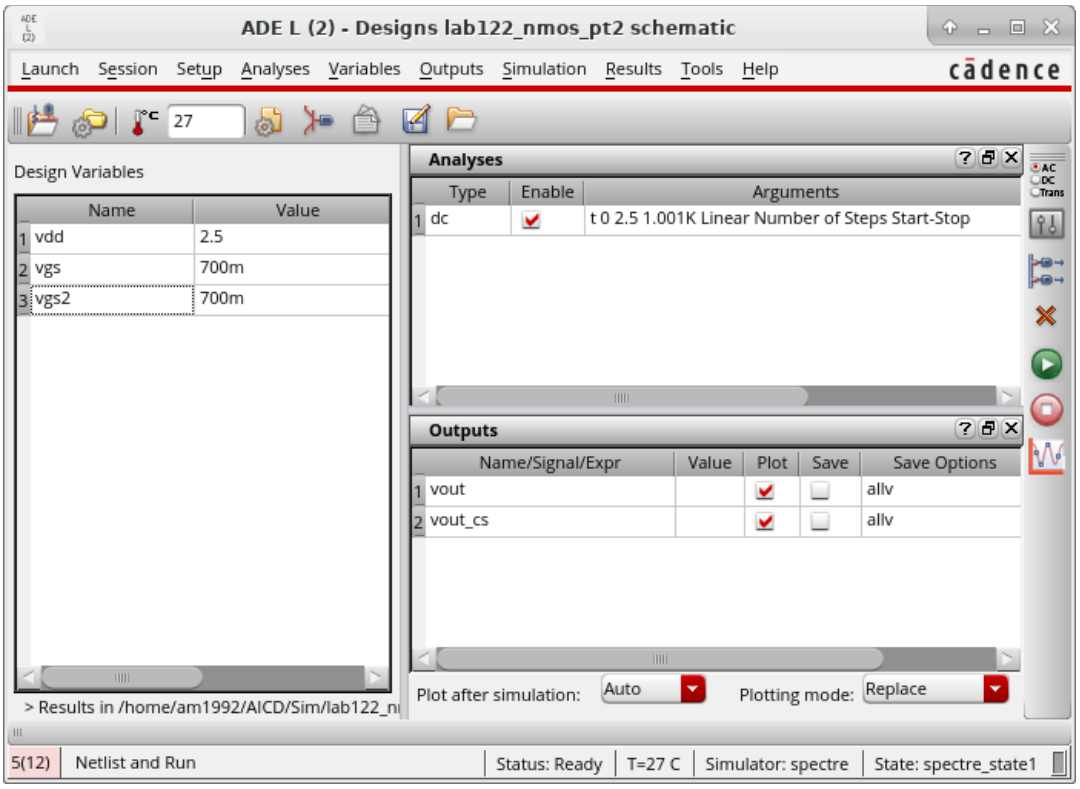
(5.4.9) If you chose to modify your existing circuit, rather than create a new schematic from scratch, you should still have your ADE set up for simulation. If not, or you have created a new schematic, launch the ADE and follow the steps in the previous section to set up the swept-DC analysis.



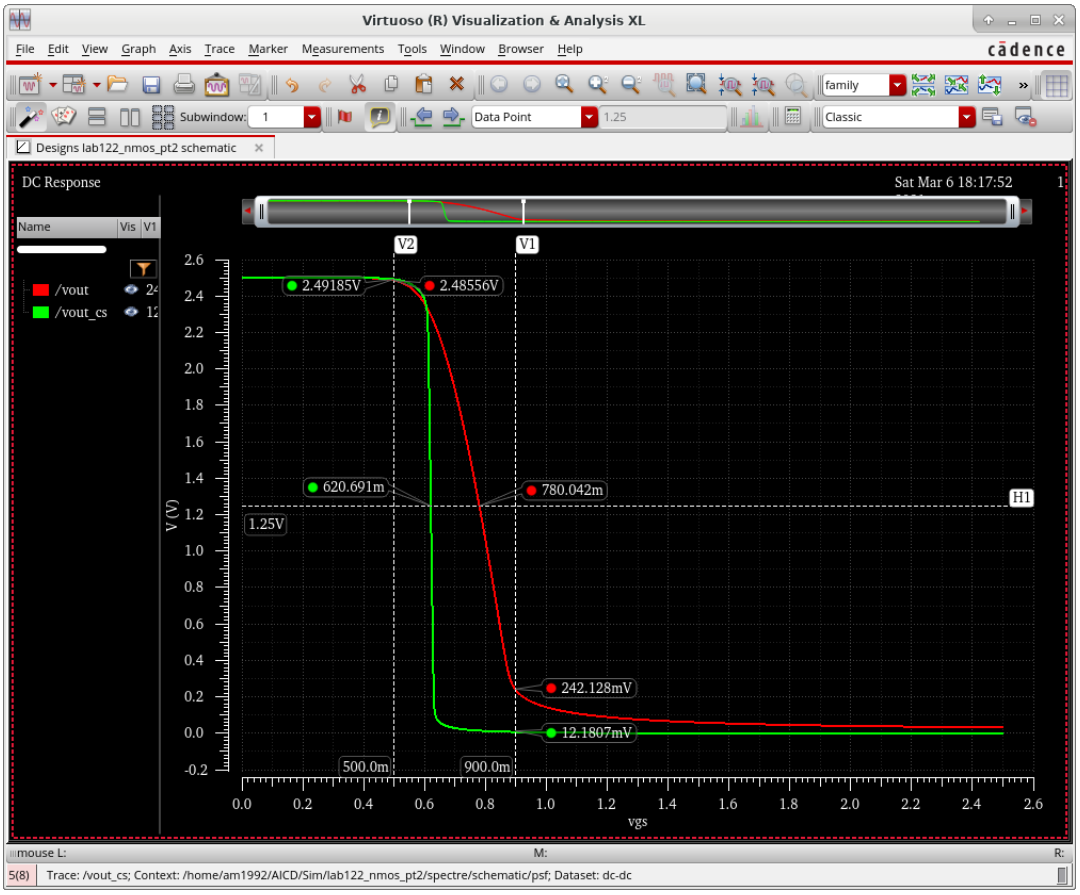
< Figure 51. The pmoshv (GPDK180) and vdc (analogLib) Add Instance Forms. >

(5.4.10) Since you have added a new design variable to your schematic (vgs2) you will need to copy it from the schematic to the ADE (**Menu: Variables à Copy From Cellview**). Initialise vgs2 to 0.7V in the Design Variables section of the ADE. You will also need to add the output net (vout_cs) of the new circuit to the Outputs section of the ADE (**Menu: Outputs à To Be Plotted à Select On Schematic**). Press the 'ESC' key to exit the plotting mode of the ADE.

(5.4.11) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. If you followed all the instructions correctly, the simulation should run properly and both a textual output window as well as a waveform output window should open. See Figure 24 (Makers have been added post simulation.)



< Figure 52. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 53. The Resistor-Loaded Voltage Amplifier Schematic as in Figure 20. >

(5.4.13) **Discussion**. If we compare the voltage transfer curves of the resistor-loaded amplifier and the current-source loaded amplifier, we see the transition of the VTC is far steeper for the latter circuit. As discussed previously, the slope of the VTC is the graphical measure of gain – from the VTC, we have a voltage gain of around $\Delta V_{OUT}/\Delta V_{IN}$ = 1V/0.01V = 100, at least a factor of ten increase over the resistor-loaded amplifier.

Referring back to part 1 of the lab, recall the graph of I_{DS} vs. V_{DS}. The small-signal resistance of the MOS device was given by the slope of the large-scale I-V curve. In the saturation region, where we operate both the PMOS and NMOS devices for amplifier design, when $V_{DS} \geq V_{GS} - V_T$, the I-V curve becomes horizontal and (in the ideal case) independent of increasing drain-to-source voltage. This infers a very-high drain-to-source resistance (represented by r_o in the small signal model of the MOSFET) with $\Delta I_{DS}/\Delta V_{DS} = 0$ (or equivalently $\Delta V_{DS}/\Delta I_{DS} = \infty$). The key difference, however, between the resistive and PMOS loads is that (as you can observe from the graphs) the I-V curves of the PMOS device (specifically in the saturation region of operation) do not need to pass through the origin. That is, the high impedance of the PMOS load is independent of the DC bias level of either IDS or VDS. We have decoupled the setting of the gain from the setting of the DC operating point.

From the horizontal marker set in Figure 24, we can see from the output plot what the required VGS for the NMOS device should be for the correct current demand for VOUT = VDD/2. For the 20KΩ resistor-load a VGS of ~0.78V and for the current-source load a VGS of around ~0.62V will give both amplifier configurations a VOUT of around 1.3V (~VDD/2). We will make use of these values in the next section – when we simulate the time-domain responses of both circuits.

We can think of the current source loaded circuit in terms of trying to balance the current being 'sourced' by the PMOS device and the current being 'sunk' by the NMOS device. The only time the output VOUT will be mid-rail (VDD/2) is if the current sourced by the PMOS is the same as the current sunk by the NMOS. There are two scenarios if the sourced/sunk currents are not equal. (1) if the current sourced by the PMOS is greater than the current sunk by the NMOS, the excess current will be pushed through the very high output resistance of the NMOS and the output voltage will be forced up to the supply rail. (2) If the current sunk by the NMOS is greater than the current sourced by the PMOS, the excess current will be pulled through the very high output resistance of the PMOS and the output voltage will be forced down to the ground rail. Note that the difference between the currents being sourced (PMOS) and sunk (NMOS) will be influenced by different carrier mobilities of the two types of device. The PMOS device may be scaled (W/L ratio) to compensate for its lower carrier mobility.

We also note that with the PMOS load, V_{OUT} now extends from the power rail (2.5V) closer to the ground rail (0V) due to the much larger PMOS drain-to-source resistance. In this case, the residual resistance of the NMOS device is much smaller, where comparably less voltage is dropped across it (than when compared to the 20KΩ resistive load).

In the next section we will simulate the time-domain (transient) responses of the voltage amplifier circuits.

5.5 Time-Domain Response of the Single-Stage NMOS FET Amplifier

(5.5.1) We will now carry out a transient simulation of both the resistor-loaded and current source loaded single-stage NMOS FET voltage amplifiers. We will apply a DC bias (as specified above) to the gate of each NMOS transconductance device to set the DC operating point V_{OUT} = V_{DD}/2. We will apply a 10mV sinusoidal perturbation to the input and observe the output.

(5.5.2) In the schematic editor window, replace the gate-to-source DC voltage sources (**vdc**) of the NMOS FET transconductance devices with a time-varying Sinusoidal voltage source (**vsin**). Leave the gate-to-source DC voltage source of the PMOS unchanged.

Recall, you can use the **Delete** command to delete the **vdc** part and add a new instance of the **vsin** part. Or you can change **vdc** to a **vsin** component directly by editing its properties (**Menu: Edit à Properties à Objects**, or **Edit Properties** icon-button, or press '**Q**') and changing the cell name from vdc to vsin. You can also select the part, right-click and choose Properties from the context menu. The 'Add Instance' form will then change from a vdc form to a vsin form.

(5.5.3) For the **resistor loaded amplifier**, set the **vsin** attributes as follow: ‘DC Voltage’ = vgs, ‘Amplitude’ = 10mV, ‘Offset voltage’ = 780mV, and ‘Frequency’ = 10KHz. OK the form.

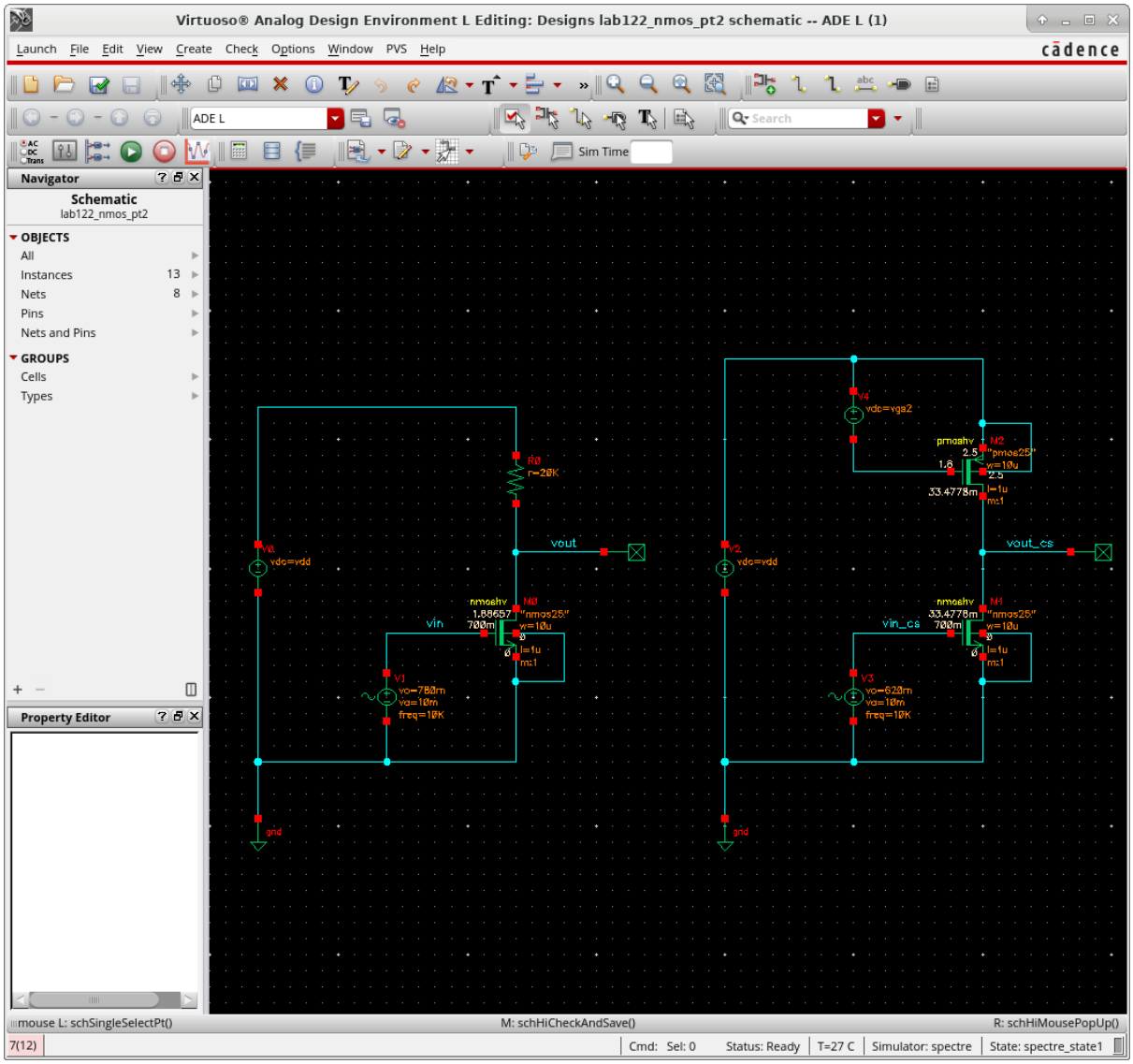
(5.5.4) For the **current-source loaded amplifier**, set the **vsin** attributes as follow: ‘DC Voltage’ = vgs, ‘Amplitude’ = 10mV, ‘Offset voltage’ = 620.7mV, and ‘Frequency’ = 10KHz. Ok the form.

(5.5.5) Name the input nets of the two amplifiers *vin* (for the resistor-loaded amplifier) and *vin_cs* (for the current-source loaded amplifier). See Figure 25.

(5.5.6) **Check and Save** the schematic. There should be no errors or warnings. Use the CIW to diagnose any errors or warnings (if there are any).



< Figure 54. The vsin Edit Object Instance Form. >

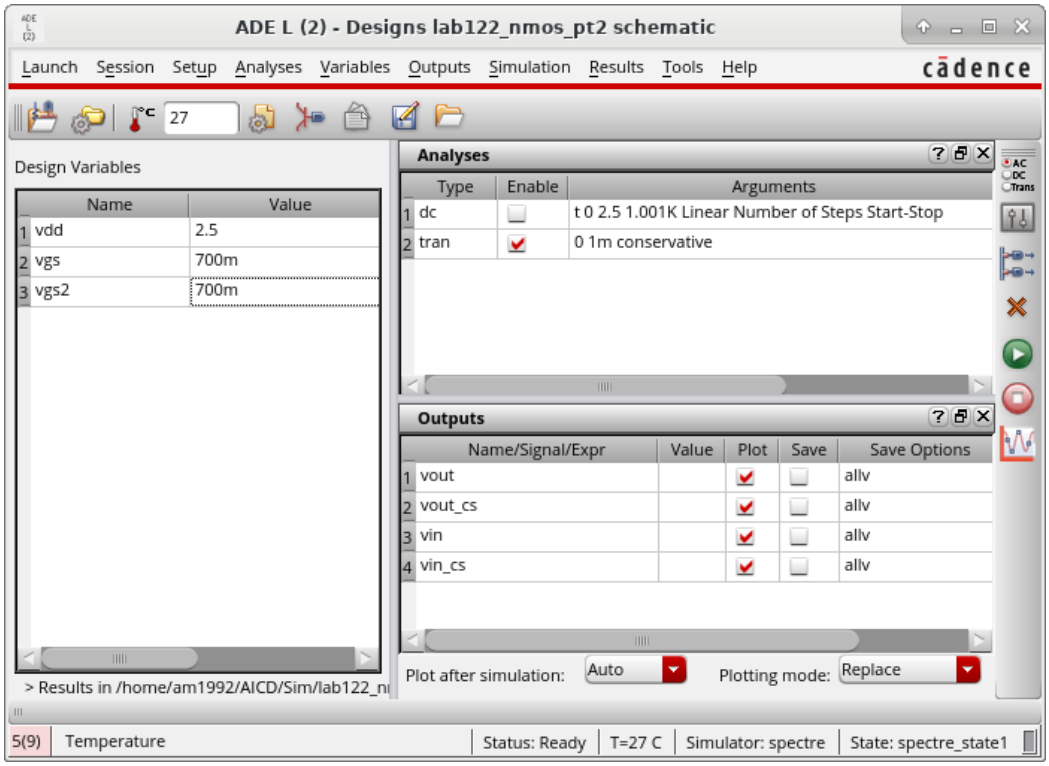


< Figure 55. The voltage amplifier circuits with vsin excitation. Amplitude = 10mV, Frequency = 10KHz, DC Offset = 780mV & 620mV. >

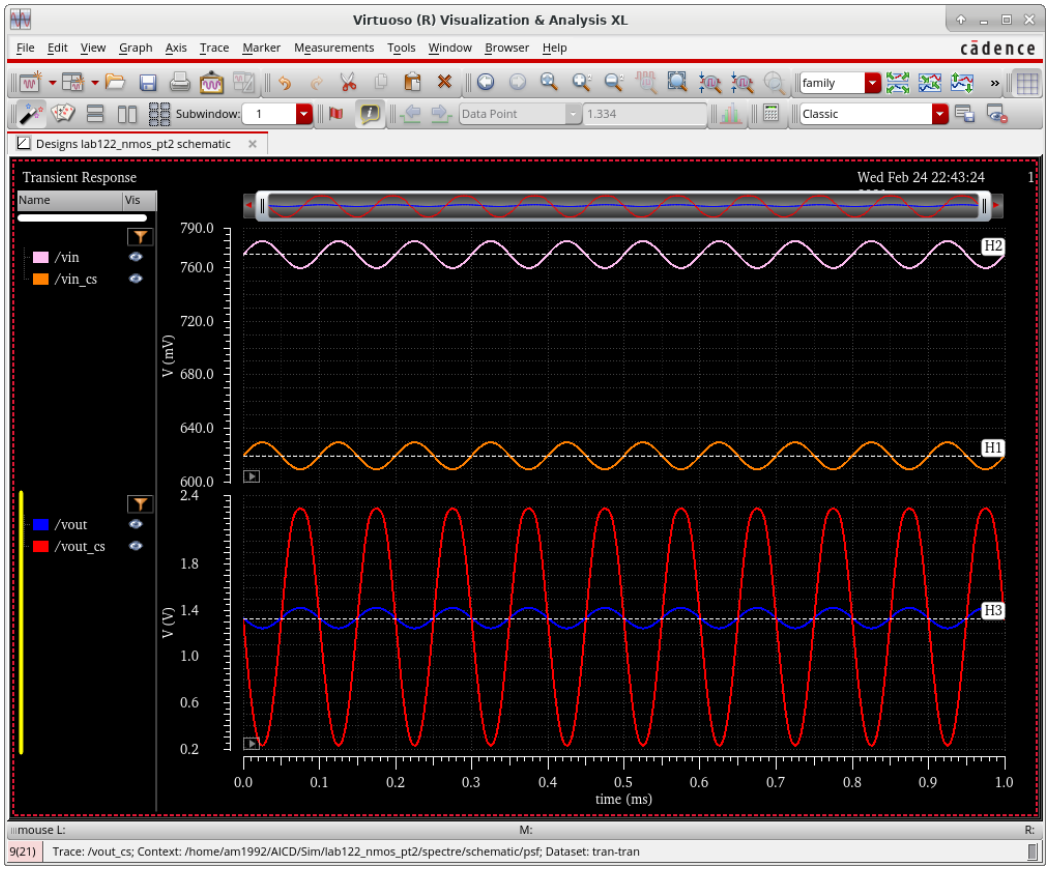
(5.5.7) We will add a transient analysis to the ADE Analyses list. From the '**Choose Analyses**' form, select the '**tran**' option button and enter '1ms' in the 'Stop Time' text field. Set the 'Accuracy' option button to 'moderate'. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. OK the form. And the **tran** analysis should appear in the 'Analyses' section of the ADE window next to the DC Analysis. Turn the DC analysis OFF by unchecking the 'Enable' option.

(5.5.8) Add the two input nets – *vin* and *vin_cs* – to the Outputs list section of the ADE window to be plotted. You should already have *vout* and *vout_cs* in the list – if not, add them.

(5.5.9) Run the **Netlist and Run** command by clicking on the green play button. If the simulation runs correctly, you should see the text output window as well as the waveform output window as shown in Figure 28.



< Figure 56. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 57. The waveform output window with horizontal markers. >

(5.5.10) **Discussion**. In Figure 27, the input signal (top) and output (bottom) waveforms have been separated into their own graph spaces. Note that the input signals are the same for both resistor-loaded and current-source loaded amplifiers except for the DC input bias/offsets -780mV and 620.7mV respectively. From the output waveforms we can see both are riding on a VOUT ~ 1.25V (approximately VDD/2) satisfying the maximum swing requirement. The output waveform of the 20kΩ resistor-loaded circuit (blue) is considerably smaller in amplitude than the output waveform of the PMOS current-source loaded circuit (red).

5.6 Plotting Modes of the ADE

(5.5.1) Finally, notice at the bottom-right of the ADE window, under the Outputs section, there are two drop down menus: **Plot after simulation** and **Plotting mode**. Both of these options together determine the behaviour of the waveform output window (the Virtuoso Visualisation & Analysis XL tool by default) after running a simulation.

(5.6.2) From the ADE user guide. ‘Plot After Simulation’ menu options:

***Auto**: Automatically plots the outputs after a simulation is run. This is the default option. For every subsequent run, the new graph replaces, appends, or opens a new (sub) graph window according to the Plotting Mode option selected. Every time a graph is plotted, the default settings are used.*

***Refresh**: Updates open graphs in the output waveform window with new simulation results and maintains graph and trace settings. If no graph is open, the results are plotted in a new graph window and in subsequent runs, outputs are refreshed in the same graph. The Refresh option ignores the modes set in the Plotting Modes list.*

***None**: Does not plot output. You have to manually invoke the waveform output widow using the Plot Outputs button*

(5.6.3) From the ADE user guide. ‘Plotting Mode’ menu options.

***Replace**: Replaces the existing graph with the new graph. This is the default option.*

***Append**: Appends the new graph to the existing graph.*

***New SubWin**: Plots the new graph in a new sub window of the existing graph.*

***New Win**: Plots the new graph in a new window. Keeps the existing graph unchanged.*

These plotting options only work when the Plot after simulation list is set to **Auto**.

End of Lab!